

Quantification of Cycling Training Routes by Segmentation and Algorithmic Analysis

Jack Jibb

School of Computing and Mathematical Sciences, University at Greenwich

2024-2025

Abstract

Training quality depends not only on an athlete's physiology but also on the road environment: long, uninterrupted climbs or low-traffic corridors benefit sustained efforts, whereas junctions, stops and heavy traffic fragment work and increase power variability. This thesis investigates whether the "training suitability of roads can be quantified and used to help cyclists plan better sessions. This paper presents an end-to-end framework that transforms raw GPS traces from a GPX file into a list of road segments, annotated for training suitability. An input pipeline cleans and map-matches traces to OpenStreetMap using OSRM in a manner similar to Luxen and Vetter (2011), and enriches the matched geometry with road metadata (e.g., highway class, speed limit, junction density). A standalone engine performs the route segmentation with a two pass processing algorithm. . Each segment has its metadata analysed, and then it is given a Training Suitability Score (TSS) derived from road context and observed ride dynamics. TSS has 5 components; one for each zone in the 5-zone training model. Together, these components form a practical route-planning system that prioritises the quality of training stimulus over performance ranking and competition.

Contents

Abstract	1
1 Introduction	1
1.1 Background	2
1.2 Research Objectives	2
2 Literature Review	4
2.1 Training Suitability and Segmentation Algorithm Design and Analysis	4
2.1.1 Research Objectives	4
2.1.2 Algorithmic Approaches to Segmentation and Categorization	5
2.1.3 Methodological Considerations and Assumptions	7
2.1.4 Pulling Map Data	7
2.2 Training Suitability Scoring	8
2.2.1 Research Objectives	8
2.2.2 Approaches to Suitability Scoring	9
2.2.3 Methodological Considerations and Assumptions	11
2.3 LSEPI Analysis	14
2.3.1 Legal	14
2.3.2 Social	14
2.3.3 Ethical	15
2.3.4 Political	15
3 Requirements	16
3.1 Software Requirements Specification (SRS)	16
3.1.1 Purpose and Scope	16
3.1.2 Intended Audience and Reading Suggestions	16
3.1.3 Overall Description	16
3.1.4 Specific Requirements	18
3.2 Standards and Compliance	20
4 Implementation	21
5 Design	22

5.1	Scope and Purpose	22
5.2	Architectural Design	22
5.2.1	Architectural Views	22
5.2.2	Design Rationale	25
5.2.3	Design Patterns and Styles	25
5.3	Module-Level Design	25
5.3.1	GPX-to-Enriched-JSON Pipeline Modules	25
5.4	Interface Design	26
5.4.1	User Interface	26
5.4.2	Software Interfaces	26
5.5	Data Design	26
5.5.1	Data Models	26
5.5.2	Database Schema	26
5.5.3	Data Dictionary	26
5.6	Standards and Compliance in Design	26
6	Methodology	27
6.1	Critical Reflection on Methodology	27
6.2	Component Implementation	27
6.2.1	Input Pipeline	27
6.2.2	Segmentation Engine	30
6.2.3	Segmentation Engine: Two-Pass Weighting and Recursive Merging	32
6.2.4	TSS Engine	33
6.3	Testing and Results	33
7	Results and Conclusions	35
8	Further Discussions and Research Gaps	36
A	Further Reading	39
B	Code Listings and File Structures	40
C	Directory Structure	47
D	Application Documentation	48
D.1	Segmentation Engine Documentation	48
D.2	Pipeline Documentation	50
D.3	Archived Documentation	62
D.4	Archived Map Data Documentation	66
D.5	Thesis Proposal	68
D.6	Literature Review	78

D.7 Method	84
----------------------	----

List of Figures

2.1	HMM Matching Transition probabilities mapped over actual Ground-Truth route transition graph.	6
2.2	The 7 Zone Training Model, courtesy of Indoor Cycling Association	10
2.3	My Power Curve for August, 2024. The power curve can estimate what would be considered "hard" for an athlete at a given duration. Image courtesy of intervals.icu	13
5.1	High level system architecture for the Route Segmentation and Analysis System . .	23

List of Tables

2.1	Training Suitability Scoring > Methodological Considerations and Assumptions . . .	12
3.1	Functional Requirements (FR) - Overview	19
3.2	Non-Functional Requirements (NFR) - Overview	19
3.3	Database Requirements	19
5.1	Architectural Design > Architectural Views - structured summary	24
6.1	Canonical extension keys and example synonyms	29

Chapter 1

Introduction

High-quality cycling training depends on being able to execute the intended effort without interruption. Professional riders often train on long climbs with low-traffic roads in remote locations; most amateurs, by contrast, plan routes with general-purpose mapping tools and only discover mid-ride that junctions, blind corners or traffic furniture force repeated wattage spikes or dips. The result is high power variance, which increases Normalized Power, a metric that approximates the metabolic cost of a ride according to Coggan and Allen (2012). This means that during an endurance or structured interval session, an athlete may not be able to target the power zones they are aiming for, leading to suboptimal training stimulus. Most mapping tools do not consider the suitability of a road for structured training, focusing instead on either performance rankings (e.g., Strava segments) or commuting efficiency (e.g., Google Maps). This dissertation addresses that gap. by investigating whether the suitability of a road for structured training can be quantified from just ride data and open-source mapping solutions so that routes can be planned with an optimal path; maximizing segments that support sustained efforts. We refer to this property as *training suitability*. The proposed system would ingest raw GPX traces, perform map-matching against OpenStreetMap with OSRM's match engine, enrich the resulting paths with road metadata (e.g., highway class, speed limits and junction density), and then apply a segmentation procedure. Segments are identified using a multi-pass algorithm, that takes each "leg" of the route (a leg being the smallest unit of a route, returned by OSRM's match engine) and attempts to merge every consecutive leg recursively until either there is one leg, or the merge cost of each segment/leg exceeds a threshold. This is known as a merge-cost algorithm, classed as a greedy algorithm, since it always chooses the best option at each step, without considering future consequences. Each segment then has a set of scores generated from the road and GPX metadata, one for each accepted training zone below 1 hour, defined by Sage (2019). This set of scores is collectively known as the Training Suitability Score (TSS). To support this goal, I aim to implement (1) batch processing for large GPX collections, (2) segment de-duplication and matching so new routes can be expressed as sequences of known segments if available, and (3) a minimal web application for exploring routes, visualising segment scores and gathering feedback. In summary, this work aims to move route planning beyond performance rankings towards the deliberate selection of roads that maximise training stimulus. The

remainder of this chapter provides background and states the research objectives that guide the study.

1.1 Background

While route planning tools for cyclists abound, most focus on navigation efficiency or social competition rather than the quality of the training stimulus. Popular platforms such as Strava and Komoot allow riders to discover new roads and measure themselves against others, but their segment definitions are crowdsourced and oriented towards ranking performance on short stretches. Navigation services (e.g., Google Maps, OSRM, OSM-based planners) optimise for travel time or scenic value but do not provide insight into how well a route supports structured intervals. In the literature, map matching of noisy GPS traces to road networks has been well explored, with Hidden Markov models and Viterbi decoding achieving high accuracy (Newson and Krumm, 2009). Work on ride classification has also examined the use of power metrics like Normalised Power and intensity factors to characterise efforts (Hurley, 2020). However, there is a gap in combining these techniques to produce a data-driven assessment of road suitability for different training zones. This dissertation builds on the existing foundations by proposing a segmentation and scoring framework that prioritises sustained power delivery and low interruption frequency, aiming to bridge the gap between navigation and training science.

1.2 Research Objectives

This dissertation investigates whether the *training suitability* of roads can be quantified from ride data and open map sources to support route planning for structured efforts. The objectives are:

- O1. **Define 'training suitability'.** Formalise a segment-level construct that analyses the training performance and route features at five levels of training suitability.
- O2. **Build a reproducible map-matching pipeline.** Establish a deterministic GPX $\rightarrow$ OSM route pipeline that normalizes the data to a standard map-matched format, using OSRM's (2021b) match service to remove unwanted GPS noise.
- O3. **Develop segmentation methods.** Compare candidates for route segmentation:
 - O3a. curvature/heading and grade change-points,
 - O3b. speed-variance and stop-density cues,
 - O3c. multi-scale wavelet (Sharifzadeh et al., 2005) change detection,
 - O3d. topology-aware separation (junction density, crossings),
 - O3e. greedy vs. dynamic-programming formulations.

-
- O4. **Engineer segment features.** Design a feature schema; including but not limited to: length, elevation gain, mean/variance of grade, curvature, stop/junction density, speed-limit, highway class, surface/cycleway tags, and simple traffic proxies derived from OSM metadata.
 - O5. **Specify a TSS model.** Create (1) a transparent rule-based baseline mapping features to a scalar training suitability score, as well as (2) an algorithmic signal analysis for each training zone (Endurance, Tempo, Threshold, VO₂max, Anaerobic/Neuromuscular) from the segment features.
 - O6. **Ground-truth construction.** Derive labels from ride dynamics (e.g., proportion of time in target zone, pause frequency,
 - O7. **Validate alignment with training intent.** Test whether TSS correlates with: (a) sustained power feasibility, (b) low interruption frequency, (c) subjective suitability ratings based on existing known training rides at given intensity.
 - O8. **Segment library operations.** Deduplicate, index, and match new routes to an evolving segment set; assess match precision for each call.
 - O9. **Human feedback (optional).** Prototype lightweight feedback (thumbs-up/down or pairwise choices) and test whether incorporating it improves ranking metrics.
 - O10. **Reproducibility and privacy.** Ensure runs are reproducible and only anonymized segment scores are stored to the database (no user-identifiable ride data).

Chapter 2

Literature Review

A literature review was conducted¹ to determine the current state of the art in route segmentation and suitability scoring. The review was conducted over an extended period of time, pre-dating most of the experimentation and development of the system. While a lot of the literature was not directly implemented into the system, it was used to inform the design and implementation, as well as provide a springboard for critical thinking about how the system could be designed.

2.1 Training Suitability and Segmentation Algorithm Design and Analysis

2.1.1 Research Objectives

The purpose of creating an algorithmic approach to categorizing and segmenting a GPX route is to be able to construct new routes based on the requirements of the rider's training plan. Intuitively, it makes sense to segment a route; riders do not have to analyse every part of a map in street view to determine if it is viable for their specific workout. Riders are specifically interested in sections of the route such as climbs, descents, busy roads, and gravel. The objective of research should be to come up with a method of segmentation, and subsequent categorization, of road segments so riders can path their ride to maximise the TSS of their route. Intuitively, we could segment a route by the roads that it follows. When a route changes to a new road, the route can be segmented. However this can prove more difficult to formalize, as a road can have many intersections, and a logical route could span two roads. The question becomes how to define a split so that segments occur at meaningful points, and how do we define meaningful?

¹Note to reader: since the literature review was conducted in advance to any other section in this paper, any method mentioned earlier was considered to be effective as a result of this literature review

2.1.2 Algorithmic Approaches to Segmentation and Categorization

There are several methodologies when it comes to segmentation, including (but not limited to) clustering, heuristic methods, signal analysis, and map-based segmentation. Some, or all of these may be applied to the algorithm, since there is not one method to solve all segmentation types. What follows is a review of the literature that exists that could contribute to achieving advanced segmentation.

Paul Newson and John Krumm: "Hidden Markov Map Matching Through Noise and Sparseness" A useful way to clean up the location data as a pre-processing step could be through map matching. Newson and Krumm (2009) explore map-matching through a HMM model on timestamp-marked latitude and longitude pairs. The approach showcased in the paper is focused on automobile traffic, so there may be some need to tweak the emission delta for bikes, to accommodate for larger variance in the horizontal travel distance, as well as increasing the search range for nodes around each track point. The search would then use a Gaussian probability curve, centred on GPX trackpoint, with nodes closer being more likely than ones further away. Then a transition probability will be implemented between consecutive points so straight line distance is more probable. Lastly, a Viterbi algorithm will be run to find the "most likely" node sequence. Their approach includes two variables that need to be tweaked to maximise accuracy. This could be done via machine learning, but is out of the scope of this research. It will be set to a "good enough" setting. Once the node sequence is determined, the sequence of ways can then be determined. At first I considered a rough implementation of this algorithm for my purposes, keeping it lightweight but targeted for my use case. However, after researching my next source, I realized that the OSRM (Open Source Routing Machine) project already implements a map-matching algorithm that is more than sufficient for my needs, and had a HTTP interface, so I can simply send the GPX data to it, and it will return a JSON object with the matched route.

HMM-based Map Matching with OSRM OSRM's `/match` service implements the HMM paradigm, returning one or more matched sub-traces with per-point confidence (Project OSRM, 2021b). Telenav's (2018) OSRM notes explicitly discuss HMM+Viterbi for map matching. Practically, OSRM's design makes this workflow reproducible and customizable: you can run the engine locally in Docker for deterministic experiments and adjust routing behaviour with Lua profiles to better reflect cycling vs. driving, surface penalties, or junction costs before (re)extracting the graph used by the matcher (Project OSRM, 2021a, 2025a,b). Taken together, this lets us: (1) map-match noisy rides, (2) export rich annotations (nodes/way/gpx) for downstream segmentation, and (3) tune profiles to reflect training-use contexts (e.g., prefer longer, open roads, but avoid major highway links), then re-run match at scale.

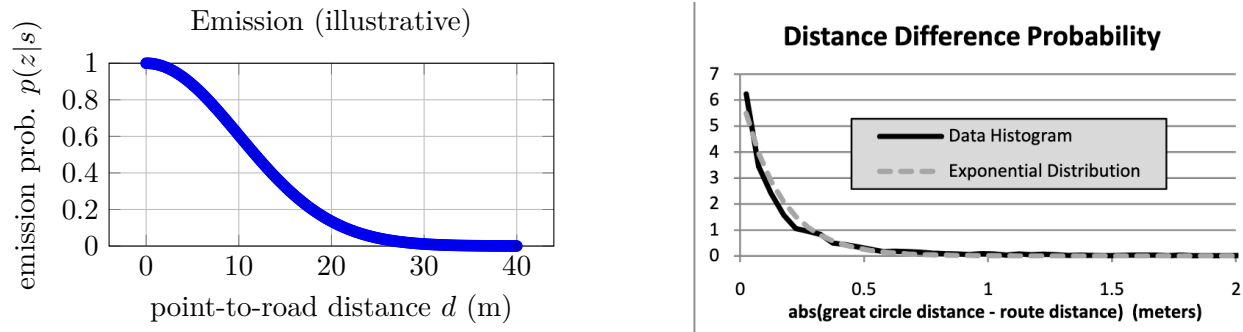


Figure 2.1: HMM Matching Transition probabilities mapped over actual Ground-Truth route transition graph.

Customising behaviour via Lua (before (re)extract).

```

1 -- Inside a custom profile (simplified example)
2 local function way_processing(way, result)
3     local highway = way:get_value_by_key('highway') or '
4     local surface = way:get_value_by_key('surface') or '
5     local smooth  = way:get_value_by_key('smoothness') or '
6
7     -- Base speeds (km/h) - tune to your use case
8     local base = { residential = 22, tertiary = 20, secondary = 18, primary = 16 }
9     result.speed = base[highway] or 15
10
11     -- Discourage trunk and trunk_link for training rides
12     if highway == 'trunk' or highway == 'trunk_link' then
13         result.speed = result.speed * 0.30
14         result.weight = (result.weight or 1.0) * 3.0
15     end
16
17     -- Prefer paved+smooth; discourage unpaved
18     if surface == 'unpaved' or surface == 'gravel' then
19         result.speed = result.speed * 0.75
20         result.weight = (result.weight or 1.0) * 1.4
21     end
22     if smooth == 'excellent' or smooth == 'good' then
23         result.speed = result.speed * 1.10
24     end
25 end

```

Listing 2.1: Lua profile example snippet: discourage trunk links, prefer residential/smooth paved for cycling

Boeing, G: "Modeling and Analyzing Urban Networks and Amenities with OSMnx"

Boeing (2017) details a novel Python package, `osmnx`, that downloads and analyses street networks from OpenStreetMap, which implements simple query language for pulling street data from the OSM database. It also includes tools for simplifying OSM map data so ways and nodes represent roads and intersections. As described in their Getting Started guide, the library enables "a programmatic

approach to visualizing OSM data as a mathematical graph". The graph can be formatted as a `GeoDataFrame`, which can be exported using `networkx` and `geopandas` libraries. Using a function like `graph_from_bbox()`, it is possible to obtain a road network around the bounds of a GPX track by getting the maximum and minimum latitudes and longitudes, which can then return a clean and topologically correct graph $\mathcal{G}$ to map match to.

2.1.3 Methodological Considerations and Assumptions

Firstly we must obtain a GPX file. This is given from either a new or existing activity. It can be obtained in many ways, either from common fitness websites (Strava, Garmin, MapMyRide, etc.), or directly downloaded from a GPS device. Either way, there are some standards we can take advantage of. Each GPX is an XML-style file. An example can be found in Appendix B.9. It is obvious that the GPX file follows a hierarchical tree structure, with metadata including time, GPS coordinates, and cycling-specific physiological metrics. Not all of the data is named consistently, especially in "extensions", so it is important to consider variations in naming. This can be normalized during the processing with a regex-style dictionary map, having a standard name for each useful metric, along with a list of synonyms.

Once downloaded, GPX data must be normalized and processed. This can be done by converting to a standardized JSON object structure, where each object contains at least Latitude, Longitude, Elevation, and timing data. After this, erroneous values should be corrected without removing time series data. This can be done with either flagging or interpolation. Flagging is useful for detecting missing data, such as 0s or NaNs, which can be ignored in the data processing, but removing the data would cause timing mismapping, putting the data signals out of sync. Simply detecting 0s or NaN values in the data stream is sufficient for some datapoints, but the preferred approach for numerical data is to approximate it with interpolation of the average of the surrounding present values. Lastly, derived metadata, such as speed or heading, should be calculated mathematically, and added as an additional extension in the JSON file to reduce processing time on any non-concurrent processes.

2.1.4 Pulling Map Data

OSMnx allows easy and efficient pulling of API data from the OpenStreetMap database. This could be used to correct the GPS data, and also to enrich the data for segmentation and categorization. To do this, we use the `os.graph_from_bbox()` function to pull only nearby data (within the bounding box of the route). The approach is as follows:

1. Find the largest and smallest coordinates in the route
2. Generate a bounding box of the extreme coordinates
3. Send the bounding box to a function that pulls the data from OSM Overpass API
4. Download the map as a `.osm2` file for caching.

²Important to note: `.osm` files are an XML-style map file that is downloaded directly from the internet. OSRM

2.2 Training Suitability Scoring

2.2.1 Research Objectives

To define a training suitability score, we must first define what a training suitability score is. This is a subjective measure of how suitable a route is for a specific targeted training zone. We must define what the zones are, define the scale of our score, define a method of obtaining said score, and define a validity checking function to verify that the score isn't affected by outlier conditions (such as noise, riders getting punctures, or other issues unrelated to the route itself). Since the aim is to build a database of scores, each score must be isolated and normalised to the rider. So absolute power could not be used as a good metric, since power thresholds are different per rider. Instead, we must consider rider-agnostic metrics. We also need default metrics, that don't relate to data that may not be present (such as extensions). We should be able to generate a score with nothing but base GPX data (latitude, longitude, elevation, time) and OSM route data, with the ability to refine the score with metrics like heart rate, power, and cadence.

Algorithm Design

1. **Model family:** Compare transparent formula-based models (weighted feature sum with calibrated bins) vs. learned models (ordinal regression / pairwise ranking).
2. **Weight learning:** Learn weights from coach labels or pairwise "A more suitable than B judgments; constrain for interpretability.
3. **Multi-objective scoring:** Support per-intent heads (Endurance/Threshold/Neuromuscular) sharing a common feature backbone.
4. **Uncertainty:** Output a confidence/uncertainty score (e.g., via bootstrap or ensembling) for downstream UI.

Validation and Evaluation

1. **Reliability:** Test consistency on repeated rides (ICC, coefficient of variation) under GPS noise and sampling-rate changes.
2. **Criterion validity:** Correlate TSS with expert/coach ratings and with proxy physiology (normalized power,
3. **Discrimination:** AUC/average precision to separate segments intended for different training zones.
4. **Calibration:** Reliability plots (expected vs. observed difficulty bins).
5. **Generalisation:** Cross-validate across geographies (urban/rural, hilly/flat) and seasons; check domain shift.

doesn't accept this as a file type, so it is necessary to use something like Osmium for conversion from .osm→.pbk.

Robustness and Edge Cases

1. **GPS/Map-matching noise:** Stress-test with perturbed trajectories; target $\leq 5\%$ score drift.
2. **Segment length effects:** Ensure fairness across short vs. long segments (length-aware normalization).
3. **Outliers:** Downweight extreme rides (e.g., device glitches, group sprints) to protect segment baselines. Can filter out data points above certain standard deviation value or absolute ceiling (2000+ watts/200rpm/250hr, etc...).

2.2.2 Approaches to Suitability Scoring

Sharifzadeh et al. "Change Detection in Time Series Data Using Wavelet Footprints"

An interesting approach to Suitability scoring relates to Fast Fourier Transforms, and a convolution approach comes from Medhi Sharifzadeh et al (2005), from the University of Southern California, introducing a concept known as "wavelet footprints" as a compact, multi-resolution approach to representation of spatial-temporal trajectories. Wavelet footprints are a more granular version of the Wavelet Transform, which in turn is a version of the Fourier Transform. The approach involves using Wavelets to transform a signal into the "Wavelet Domain". The smaller the wavelet, the more reactive it will be to change in the original signal, so by adjusting the size of the wavelet, the signal can be filtered to be more or less reactive to change. Wavelet footprints have an advantage over the general Wavelet transform, where they only retain the most significant components. This is done by having wavelets occur in orthogonal sets. Due to Heisenberg's Uncertainty Principle, it is impossible to perfectly describe both the frequency content of a signal and the location in time of the signal. Time-domain and Frequency-Domain analysis in this regard are at opposite ends of the spectrum, but the Wavelet Domain sits in the middle, allowing for a sliding scale value, where a larger scale gives more frequency and less time resolution, while smaller scales give less frequency, and more time resolution.

The advantage of using wavelet footprints over the Fourier Transform means that it is possible to isolate points in the signal where significant changes occur in specific metrics, or combination of metrics, allowing flexibility in choosing what conditions must be met to enact a segmentation.

Indoor Cycling Association: "How Much Time in the Red Zone?" This article by Indoor Cycling Association (n.d.) details a breakdown of the 7-training-zone model, which provide a good template for cutoff times for tuning Wavelet Footprints for analysing the GPX signal. The general consensus is having 5 zones is a good compromise between continuous training definitions (specific power numbers) and binary (hard or easy). While the article describes 7 zones, Zones 1-3 all fall outside of the hour range, which for the purpose of segment analysis would be fairly useless. The 5-Zone model approach has good suitability to wavelet analysis, since small scale values for a wavelet would detect short burst efforts, while larger scales will detect longer, sustained effort. Having 5 zones allows for a reduced scale array, contributing to higher performance. A breakdown of 7 zones

is visible in Figure 2.2, courtesy of the Indoor Cycling Association, breaks down each zone. Zones 3-7 will be used for Wavelet Analysis.

 Power and Heart Rate Training Zones and RPE Chart							
ZONE	Name/Purpose	%FTP	%LTHR	RPE	RPE Description	How Does it Feel?	Duration
7	MAXIMAL	>150	n/a	10	Maximal, Explosive power	Gasping for air, can't say one word	5-20 seconds
6	Anaerobic Capacity	121-150%	n/a	9	Very, very hard	Breathless, ragged breathing, talking impossible. Severe sensation of leg effort.	30 seconds to 3 minutes
5	VO2 Max	106-120%	>106%	8	Very hard	Cannot talk, laboured breathing. Strong sensation of leg effort	3-8 minutes
4	Lactate Threshold	91-105%	95-105%	6-7	Hard	Deep forced breathing, but still sustainable. Moderate to greater sensation of leg effort.	10-60 minutes
					Moderately hard	Deep breathing; talking is very challenging	
3	Tempo	76-90%	84-94%	4-5	Somewhat hard	Heavier but rhythmic breathing, greater sensation of leg effort.	1-3 hours
					Moderate	Talking becomes uncomfortable.	
2	Aerobic Endurance	56-75%	69-83%	2-3	Easy	Light rhythmic breathing. Can maintain for hours.	Many hours
					Very easy	Can talk in complete sentences	
1	Active Recovery	<55%	<68%	<2	Very, very easy	Restful breathing; can sing.	All Day

Modified from Andrew Coggan Power Training Levels

Figure 2.2: The 7 Zone Training Model, courtesy of Indoor Cycling Association

Sean Hurley: "Normalized Power (Hurley, 2020): What It Is and How to Use It"

TrainerRoad writer Sean Hurley provides a useful and concise definition of Normalized Power (NP), a metric invented by Dr. Andrew Coggan in his book *Training and Racing With a Power Meter*. NP "reflects the disproportionate metabolic cost of riding at high intensity, by weighting hard efforts and deemphasizing periods of easy spinning", according to Dr Coggan. Essentially, NP approximates what power a rider could have put out for the same effort, if their effort was steady-state. While it is not a completely accurate metric for effort, it is a good indication of power variability, and as such is useful in determining the type of effort of a segment. The algorithm for determining NP is as follows:

1. Calculate a rolling 30 second average power for the duration
2. Raise each rolling average value to the fourth power.
3. Determine the average of all the rolling values
4. Take the fourth root.

For an input power signal, $P(t)$ over interval $(0, N)$, the formula for Normalized Power (NP) is:

$$NP = \left(\frac{1}{N} \sum_{i=1}^N (\bar{P}_r(i))^4 \right)^{1/4}$$

where $\bar{P}_r$ is the 30 second rolling average power starting at point i in the power data array.

2.2.3 Methodological Considerations and Assumptions

In order to classify a segment, it is necessary to know what an athlete values in training. As a cyclist, I have used my own experience to compile this list:

- Type: What is the categorization of the road? (A, B, Service, Motorway, Bike path)
- Surface: What is the surface of the road made of? (Tarmac, concrete, gravel, dirt, wood?)
- Behaviour: How does the road act? (Straight, gradual bends, sharp curves, switchbacks)
- Safety: How safe is this route? Are there a lot of accidents, is there a lot of commercial traffic?
- Elevation: Is this route a consistent gradient, is it flat, uphill, downhill, rolling?
- Length: How long is the route in metres.

Table 2.1: Training Suitability Scoring > Methodological Considerations and Assumptions

Name/Key	Description/Value	Notes
Type	WhatA, B, service roads, motorway, bike path, etc...	is the categorization of the road?
Surface	What?armac, concrete, gravel, dirt, etc	is the surface of the road made of?
Behaviour	How Straight, bends, switchbacks, sharp turns	does the road act?
Safety	How safe is this route?	Are there a lot of accidents, or commercial traf

Implementing a summative score that represents a segment will also benefit from taking rider metrics, such as Power, Heartrate, Cadence, and Speed. As a cyclist or group of cyclists ride a specific segment, the better picture we get of the route, as the average values will be a good representation of the training suitability of the route.

- **Power:** All cyclists have different power thresholds; in other words, two cyclists may be going the same speed up a climb, but one may be going all out, and another just going easy. They may not even have the same power output. The one going easy could have a higher power output than the person going all out, depending on their weight. As such absolute power is a bad reference point for training suitability. Rather, we need to focus on power variation, as well as power curve. A power curve is the integration of all power numbers over a duration of exercise, plotted average power in watts on the y-axis, and max sustained duration for that average power on the x-axis. The graph tends to look like an exponential decay function. An example of my own personal cumulative power curve in 2024 is shown in Figure 2.3.

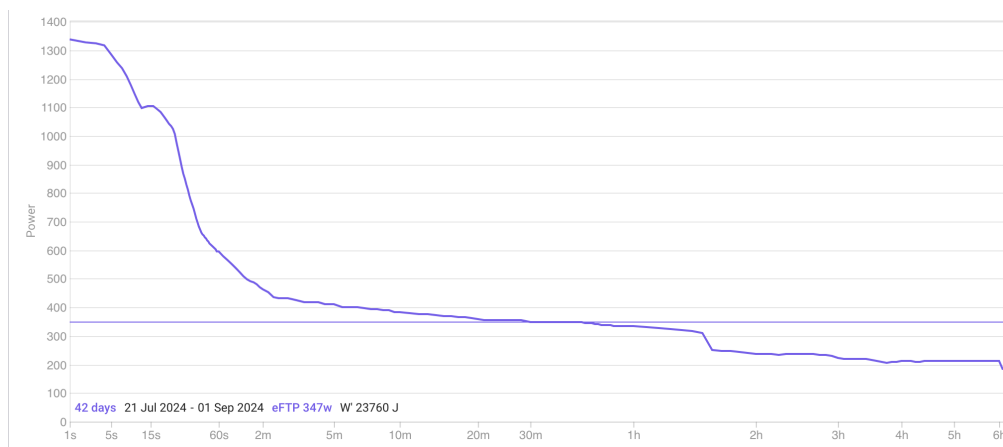


Figure 2.3: My Power Curve for August, 2024. The power curve can estimate what would be considered "hard" for an athlete at a given duration. Image courtesy of [intervals.icu](https://www.intervals.icu)

- **Route Terrain:** Hilly or technical terrain (such as lots of sharp corners or non-pavement roads) can significantly affect how good a route is for training, but in different ways. Hilly terrain could be really good for consistent efforts, if the gradient is sustained, but if there are a lot of short, steep climbs, it would be harder to maintain any consistency in effort. Likewise, gravel or dirt roads could be good for endurance if they are consistent, but throw in some sharp corners, and suddenly you have to brake a lot more, accelerate, and even focus on balancing more, which can induce more fatigue. Terrain is probably the most important metric that is intrinsic to the route itself.
- **Safety:** Safety is obviously very important in general when cycling, but it also plays a big part in performance. Having to focus on keeping safe on the road often means being ready or having to brake or slow down to avoid getting into dangerous situations. If a road can be considered "safe" (think long straight bike paths, or straight roads with very little traffic),

then the athlete is free to focus more on their effort. To consider safety is a complicated problem, since there are many aspects. One method to consider could be to come up with a points system, and apply danger points as a suitability metric. Every "unsafe" property of a road can add danger points to the segment, and a subjective scoring system could be put in place initially, and in the future, a machine learning approach could be used.

1. Safety (a composite metric defined by various factors, could also be gathered collectively from rider feedback)
2. Difference between Average and Normalized Power (in Watts/Kg)
3. Length (of segment)
4. Elevation Gain (or loss)
5. Road Quality
6. Absolute Normal Distance Change (aka how twisty the road is)

Supplementary Techniques for Segmentation and Scoring

In addition to the core segmentation and scoring methods described above, a number of supplementary techniques from the signal processing and trajectory analysis literature informed the system design. These approaches are not directly implemented in the prototype but were reviewed to provide theoretical grounding and potential directions for future enhancements:

Resampling from time to distance. GPS trajectories are often recorded at equal time intervals, which can produce highly variable spatial resolutions along climbs, descents or stops. Resampling a trajectory to equal spatial steps, sometimes called rediscrretization, interpolates intermediate points so that each successive sample represents the same distance travelled. In the movement ecology community this is known as the BovetBenhamou method and is implemented in the `TrajRediscrretize` function of the `trajr` package. Choosing an appropriate step length is critical: too short causes oversampling (high autocorrelation) and too long causes undersampling and loss of detail³.

Shortest arc and distance calculations. Calculating distances and headings along the earth's surface uses great-circle geometry. The haversine formula computes the length of the shortest arc between two latitudelongitude pairs on a sphere, while other algorithms (such as Vincenty) refine this for ellipsoids. In road segmentation this formula converts coordinate differences into accurate segment lengths and heading changes⁴.

³Rediscrretization methods like those of Bovet and Benhamou interpolate points so that consecutive samples are a fixed distance apart, thereby equalising spatial resolution at the cost of smoothing high frequency variation²⁹L205-L214.

⁴Great-circle distance via the haversine formula provides an accurate measure of the shortest surface distance between two geographic points and is widely used to compute segment lengths in road analysis³⁵L13-L21.

Wavelets for change detection. Haar wavelets provide a simple differencing operator: applying the wavelet transform to a signal returns coefficients proportional to the change between adjacent samples. At larger scales, Haar wavelets can capture broader trends while still localising changes in time. This makes them well suited for detecting abrupt shifts in speed, grade or heading in GPS data⁵.

Hysteresis thresholding. Binary decisions based on noisy signals can flicker when a value hovers near a threshold. Hysteresis mitigates this by using two thresholds: the high threshold activates the state and the low threshold deactivates it. Only when the signal drops below the low threshold does the state change. Hysteresis has been applied in edge detection and is similarly valuable in segment classification where speed, acceleration or curvature metrics fluctuate⁶.

Root mean square (RMS) metrics. RMS values summarise the effective magnitude of a varying signal by squaring, averaging and taking the square root. Unlike peak-to-peak measures, RMS captures the sustained energy of a vibration or acceleration profile and is less sensitive to outliers. For segment scoring, RMS acceleration or lateral acceleration can quantify how rough or twisty a road segment feels⁷.

Gradient variation. Changes in road grade constrain a riders ability to maintain constant power: sustained climbs require higher power to maintain speed, while frequent oscillations between uphill and downhill force repeated gear shifts and accelerations. In transportation engineering studies, variation in road grade has been shown to limit acceleration and deceleration, particularly for heavy vehicles. Monitoring the variance of the gradient signal across a segment can therefore highlight sections with highly variable efforts⁸.

Curvature and twistiness. The curvature of a planar curve is the reciprocal of the radius of its osculating circle. For polyline routes, curvature at a point can be estimated by fitting a circle through three consecutive coordinates; its inverse radius is the curvature κ . Summing the magnitudes of curvature along a segment (or computing an RMS curvature) yields a twistiness metric, which increases with the number and sharpness of bends⁹.

⁵Haar wavelets compute discrete differences between successive samples; using multi-scale wavelets allows both fine and coarse change detection within trajectory signals39L15-L23.

⁶Hysteresis thresholding employs paired high/low thresholds to stabilise binary classifications: once a signal exceeds the high threshold it remains active until it falls below the lower threshold, reducing noise-induced toggling40L200-L208.

⁷RMS metrics compute the square root of the mean of squared values, giving an effective magnitude that smoothes over individual spikes and represents the overall energy of a signal44L1-L10.

⁸Transportation studies note that fluctuating road gradients constrain acceleration and deceleration, making sections with frequent grade changes distinct from steady climbs or flats20L255-L263.

⁹Twistiness metrics sum or integrate curvature along a road: computing curvature from the circumcircle of three successive points and aggregating it across a segment gives a measure of how winding the road is46L1-L4.

Speed and acceleration smoothing. Differentiating position to obtain speed and acceleration amplifies high-frequency GPS noise. Prior to differentiation, signals should be smoothed using filters such as moving averages, SavitzkyGolay filters, Butterworth low-pass filters or modified Kalman filters. Comparative studies found that a modified Kalman filter provided the most accurate speed and distance estimates for GPS data by balancing measurement noise and physical motion models¹⁰.

Segmenting by OpenStreetMap ways. OpenStreetMap represents roads as "ways", sequences of nodes with uniform attributes. A natural segmentation strategy is to start new segments whenever the underlying way changes or an intersection is reached. However, OSM ways can be shorter than intuitive road stretches, so systems like OSMLR group consecutive ways between major intersections into longer segments (often capped at around one kilometre). Using way identifiers or OSMLR segments as boundaries ensures that segment definitions align with map semantics and facilitate enrichment with OSM metadata¹¹.

2.3 LSEPI Analysis

2.3.1 Legal

The legal dimensions of this project revolve around data protection and intellectual property. All GPX files ingested into the system are treated as personal data because they reveal an individual's whereabouts and training habits. Consequently the project was designed in accordance with the UK Data Protection Act and GDPR: the system performs privacy trimming on the first kilometre of every ride, never stores raw trackpoints beyond the scope of the immediate analysis, and only persists aggregated segment statistics. When map data is fetched from OpenStreetMap and OSRM profiles are tuned the licences of these projects are respected: both OpenStreetMap and OSRM are permissively licensed, so derived works can be integrated into an academic system provided attribution is given. Any code reused from open-source libraries (such as `cpp-httpplib`, `nlohmann/json` or `osmnx`) is cited appropriately and the original licences are included in the repository, fulfilling our obligations under the MIT and BSD licences.

2.3.2 Social

From a social standpoint the system aims to empower cyclists of all abilities to train more effectively, but it also has the potential to exacerbate inequalities. Experienced riders with sophisticated power meters will be able to contribute richer data to the segment database, which could bias the Training Suitability Score towards routes favoured by well-resourced athletes. To mitigate this we

¹⁰Applying smoothing filters (e.g. moving average, SavitzkyGolay, Butterworth or modified Kalman) before differentiation reduces GPS noise; a modified Kalman filter has been shown to produce the most accurate smoothed speed and distance estimates for vehicle trajectories^{17L71-L81}.

¹¹OSMLR (Open Street Map Linear Reference) groups consecutive OSM ways into road segments between intersections, producing more meaningful, consistent segments (typically under one kilometre) for analysis and metric aggregation^{28L69-L90}.

deliberately designed the scoring algorithm to rely on ride agnostic features (e.g., road type, slope and curvature) and to normalise physiological metrics relative to each rider’s baseline. Furthermore, the open-source nature of the software means local clubs can host their own instances and build segment libraries tailored to their communities, reducing dependence on proprietary platforms. The interactive web interface, implemented using a lightweight stack of Flask and Leaflet, is accessible on modern browsers and does not require installing proprietary software.

2.3.3 Ethical

Ethically, the most salient issue is consent. Users must consent to their ride data being processed, and they should be aware of how long the data is retained and for what purpose. Our ingestion workflow therefore requires explicit opt-in and provides a simple mechanism for riders to delete their data. Another ethical consideration is the potential for the system to encourage risk taking: if an algorithm classifies a particular steep descent as high suitability for neuromuscular efforts, some riders might push themselves beyond safe limits. To prevent this the scoring model includes a safety term that down-weights routes with high accident records or poor visibility, and the user interface warns that the tool is a guide rather than a directive. Finally, we considered the environmental impact of running an always-on cloud service. By containerising all components and supporting local processing on commodity hardware we reduce the carbon footprint associated with large centralised servers.

2.3.4 Political

On a political level the project touches upon debates around open data and civic infrastructure. OpenStreetMap is maintained by volunteers and reflects a collective effort to democratise geographic information. By building upon this commons we contribute to the public good rather than locking data behind commercial terms. However, routing and training tools can influence cycling patterns and therefore intersect with transport policy: highlighting certain roads as training-friendly could increase cyclist traffic on them, which may require coordination with local authorities for safety improvements. We do not advocate for specific policy outcomes but we recognise that our system could inform discussions about cycle lane provision, speed limits and road design.

Chapter 3

Requirements

3.1 Software Requirements Specification (SRS)

3.1.1 Purpose and Scope

The purpose of this Software Requirements Specification is to define the functional and non-functional requirements for the GPX-Based Route Segmentation and Analysis System. This system enables end users (primarily cyclists and coaches) to upload recorded GPX ride data or draw routes on a web map, automatically segment the route into reusable "segments, match them against a central database of known segments, and compute training-relevant statistics (Training Suitability Score, TSS). The SRS covers requirements for both *upload mode* (GPX ingestion, segmentation, database updates) and *query mode* (route matching without creation).

3.1.2 Intended Audience and Reading Suggestions

- **Developers:** Read entire SRS to understand system scope, functional interfaces, and data models.
- **Testers:** Focus on Section 3.1.4 (Specific Requirements) for test case derivation.
- **Stakeholders (coaches, cyclists):** Sections 3.1.1–3.1.3 provide high-level understanding of capabilities.

3.1.3 Overall Description

Product Perspective

This system is a standalone web service that is built on a LAMP-like technology stack (Linux, Nginx, MySQL, Flask), augmented by Python and Dockerized C++ processing engines. It integrates with third party mapping libraries (OpenStreetMap, Overpass, Osmium and OSRM). Conceptually, it acts as a data processor, taking user's GPX files from their GPS-enabled devices, and processing them against a central database. The system acts as an ETL (Extract-transform-load) pipeline

for incoming GPX files, as well as a query engine for user-inputted drawn map routes from a map drawing system.

Product Functions

- **GPX Ingestion:** Accepts GPX 1.1 uploads, applies privacy-filter trimming, and parses into coordinate streams.
- **Map Data Preparation:** Fetches OSM extracts via Overpass; converts to PBF with Osmium; loads map into OSRM engine.
- **Map Matching:** Uses OSRM engine to snap GPX/drawn points to the routable network given by the .pbf file, yielding a formatted JSON object.
- **Segmentation:** Runs a two-pass C++ algorithm that (1) proposes segment boundaries by topological features and (2) matches segments in the database within spatial tolerances.
- **Database Management:** Inserts new segments (UUID + Route JSON), updates hit counts, and appends TSS values. Prunes segments below frequency thresholds after given amount of time.
- **Route Querying:** Given a drawn route, returns only the existing segments that fall along it, along with aggregated statistics.
- **User Interface:** Web pages for upload and drawing; interactive map highlighting segments and showing hover tooltips with TSS and hit count.

User Characteristics

The typical user will be of a reasonable proficiency with other online mapping softwares, such as route creation in Strava or MapMyRide. This software is mostly proof of concept, so usability considerations will be considered less important over functionality.

Operating Environment

- **Server:** Linux (Arch Linux), Docker 20+, Nginx 1.28, Python 3.8+, MySQL 8.0+, C++17 toolchain.
- **Client:** Modern web browser (Chrome, Firefox, Safari) with JavaScript enabled.
- **Network:** HTTPS/TLS for all external calls, and internal HTTP calls with GET and POST.

Design and Implementation Constraints

- **Modularization:** All components of the system must be contained within Docker containers, for easy deployment to most servers.
- **Data Formats:** GPX 1.1, GeoJSON (LineString), MySQL spatial types.
- **Standards Compliance:** Must adhere to GDPR and UK Data Protection Act; architecture described per ISO 42010.

Assumptions and Dependencies

- Overpass API availability or a local Overpass instance.
- OSM data completeness for user routes' geographic boundaries (using union of bounding boxes)
- OSRM server pre-loaded with relevant PBF extracts before running matching service.
- Users supply valid GPX conforming to standard schema. Some variance is ok, and a file with metric synonyms is accessible to the user.

3.1.4 Specific Requirements

External Interface Requirements

User Interfaces Description of UI requirements.

The user interface only requires two features:

1. The ability to upload GPX files to the server
2. The ability to map a route via any interactive map service (Mapbox, Leaflet.js or Folium (Story and contributors, 2023).py).

Software Interfaces APIs and protocols.

The system will link the front and back ends with a RESTful API. Also it is important to choose a front end framework that supports uploading multiple relatively large files (between 5-10MB each). Since a lot of the application is written in Python, Flask is a good option for this. Other API connections will be implemented to communicate with the Segmentation and TSS engines. This allows them to be hosted on separate servers in the future, to give them more processing power.

Functional Requirements

The functional requirements of the system are as follows:

Table 3.1: Functional Requirements (FR) - Overview

ID	Requirement (shall...)	Rationale/Notes	Verification
FR-01	Privacy trimming: The system shall automatically remove the first 1 km of every GPX trace prior to any processing.	Default privacy protection for user start locations.	Unit test on import; end-to-end check that no points from the first 1 km persist; code review.
FR-02	GPX import: The system shall ingest GPX 1.1 files.	Defines supported input format; reject/flag unsupported versions.	Parser tests with valid/invalid GPX 1.1 samples; schema validation.
FR-03	Containerized operation: The system shall run each component (input pipeline, match engine, segmentation, TSS, DB interface) inside a Docker container.	Portability, isolation, reproducibility.	CI builds and runs docker-compose; health checks pass.
FR-04	Data protection compliance: The system shall process and store data in a manner compliant with GDPR and the UK Data Protection Act.	Legal/regulatory requirement.	Security review; documented DPIA; automated checks for PII absence in stored artifacts.

Non-Functional Requirements

The non-functional requirements of the system are as follows:

Table 3.2: Non-Functional Requirements (NFR) - Overview

ID	Requirement (shall...)	Rationale/Notes	Verification
NFR-01	Performance: The system shall process a minimum of 1000 GPX traces per hour.	Ensures scalability for expected data volumes.	Benchmark tests; monitoring in production.
NFR-02	Reliability: The system shall have an uptime of 99.9% or higher.	Critical for user trust and data integrity.	Automated uptime monitoring; incident tracking.
NFR-03	Extensibility: The system shall allow new analysis modules to be added with minimal changes to existing code.	Facilitates future feature growth.	Code review; demonstration of adding a module in test environment.
NFR-04	Usability: The system shall provide clear error messages and documentation for users.	Improves user experience and reduces support burden.	User testing; documentation review.

Logical Database Requirements

Requirement ID	Description	Type	Priority	Source	Notes
----------------	-------------	------	----------	--------	-------

Table 3.3: Database Requirements

Software System Attributes

Reliability System components are individually testable, and hardened for failure with catch blocks for any edge conditions, with the default behaviour defined as "ignore" per section. The multi-chunk approach to processing also allows for error, without corrupting the entire dataset. If a chunk is corrupted, it can just be ignored.

Availability As a proof-of-concept system, the availability of the system is only when required for testing and presenting.

Security Security isn't as important for the proof of concept, but a lot of standard considerations can be made that relate to LAMP stack web applications:

- All HTTP endpoints *must* require HTTPS/TLS.
- Uploaded GPX files *shall* be scanned for XML schema compliance to prevent malicious payloads.
- System is interconnected on a private, secure network. Only accessible by the web host.
- Enforce TLS 1.2+, HSTS, OCSP Stapling, and complex ciphers.
- Use reverse proxy and rate limiting for connecting to the server.
- Disable directory listing, set file permissions.
- Set all security headers in Nginx.
- Input validation, normalisation, and output encoding to prevent SQL-injection and XSS attacks.
- Randomize file input names, sniff content and enforce size limits (<100Mb).
- Separate DB user per application with least privileges (no root user)
- Bind the database to localhost or internal private IP.
- Enforce strict SQL modes

Maintainability Every component of the system will be independent, and input and output formats are documented in the documentation in 8.

Portability The whole project will be made available via GitHub, and possibly a dockerized version of the system will be developed in the future for portability.

3.2 Standards and Compliance

Chapter 4

Implementation

Chapter 5

Design

5.1 Scope and Purpose

The following section is a comprehensive design plan for the Route Segmentation and Analysis System. The architecture will be described in compliance with ISO/IEC/IEEE 42010:2011 (*Systems and software engineering Architecture description*, 2011), which standardizes how systems and software architectures are documented. The System allows users to upload GPX files, or draw routes via an HTML interface. The uploaded routes are processed to identify meaningful **segments** (portions of the route that have similar road conditions). The segments are then analysed to gather training insights. Every segment will be sent to a database, where it is cross referenced with all values to see it falls within a determined margin of error ($<5\%$). If it is, then the matching database entry will have it's count increased by 1, and the training score averaged into the running average. Users can also query the system by drawing a prospective route, and will get in return a list of any known segments along the route, along with their score. The main architectural challenge in this system is to detect and match route segments directionally (direction matters), while also considering reliability.

5.2 Architectural Design

The overview of the system is presented in Figure 5.1. Present a high-level system architecture diagram using UML or SysML.

5.2.1 Architectural Views

Concept View

The architectural concept of the system can be broken down into 3 layers:

- **Presentation Layer** - This is the front end; A web application where users can either upload their GPX files, or draw a route on an interactive map. This layer will communicate with the

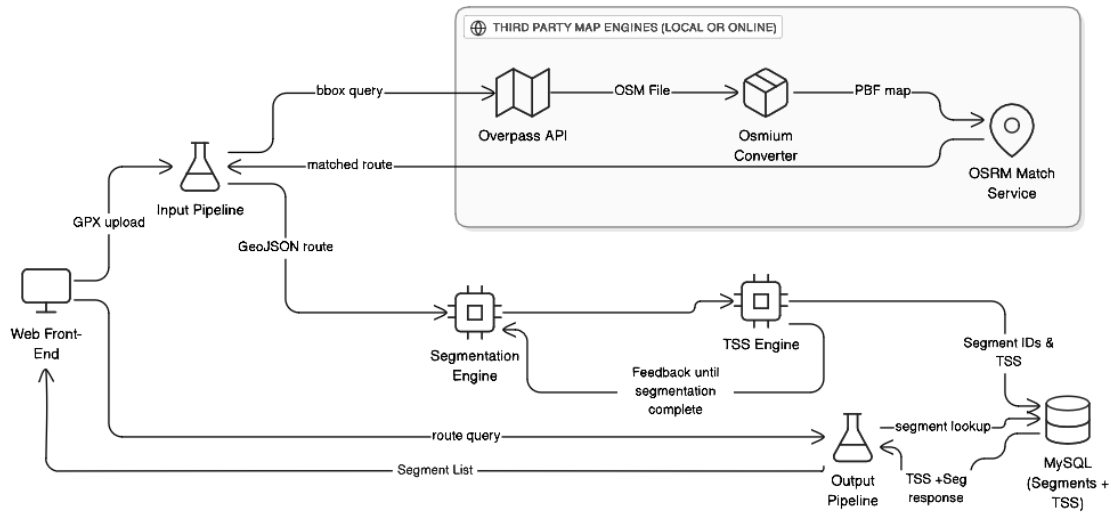


Figure 5.1: High level system architecture for the Route Segmentation and Analysis System

back end through HTTP API requests. This layer is mostly outside the scope of the project, since I am not a front end engineer.

- **Processing Layer** - The beef of the project, this includes the back-end of the system, and consists of input/output pipelines, and several processing engines. It will also contain several controllers, written in Python, to coordinate the pipelines and to send information to and from the different engines.
- **Data Layer** - This layer is represented by a SQL database that stores permanent information about route segments. External APIs also fall into this category, such as the Overpass API to fetch map data from online. These components provide necessary data but are mostly abstracted away, in the processing layer.

Functional View

The functional view includes key modules and their responsibilities.

- **Front end Web App:** Serves HTTP pages and gets information from the user.
- **Front End Controller:** Takes user information and routes it to correct API endpoint.
- **Input Pipeline Orchestrator:** Controller that handles input information routing, sending the GPX files to the pipeline, and pulling the resulting GeoJSON objects out.
- **Engine Orchestrator:** Takes routes, passes them to Segmentation algorithm, and handles the "two pass" system between Segmentation and TSS Engines.
- **DTO Orchestrator:** Formats the resulting segments, into the correct Data Object and passes it to SQL Database

- **Output Pipeline Orchestrator:** Receives the Route data from the Input Pipeline Orchestrator, validates it and sends it to Segmentation Engine. Gets segment list, and routes it back to front end in GeoJson format.
- **Segmentation Engine:** Written in C++, the engine is a specialized service that performs route segmentation and matching logic.
- **TSS Engine:** Takes a list of segments and the input GPX data, and assigns scores based on the training data that happens during each segment
- **SQL Database:** Stores all persistent segments, along with a hit count, and a list of TSS objects that have been calculated on matching segments.
- **Osmium Engine:** A docker service that runs a .osm to .pbk conversion and merging process. Used in the input pipeline
- **OSRM Matching Engine:** Another docker service that takes small GeoJson chunks, and matches them to a map network.

Table 5.1: Architectural Design > Architectural Views - structured summary

Name/Key	Description/Value	Notes
layers	itemize Presentation Layer - This is the front end	
App	Serves HTTP pages and gets information from the user. Front End Controller: Takes user information and routes it to correct API endpoint. Input Pipeline Orchestrator: Controller that handles input information routing	
Orchestrator	Takes routes	
Orchestrator	Formats the resulting segments	
Orchestrator	Receives the Route data from the Input Pipeline Orchestrator	
Engine	Written in C++	
Engine	Takes a list of segments and the input GPX data	
Database	Stores all persistent segments	
Engine	A docker service that runs a .osm to .pbk conversion and merging process. Used in the input pipeline OSRM Matching Engine: Another docker service that takes small GeoJson chunks	

Physical View

Hardware deployment and network design.

Information View

Data Flow Diagram, and Database Schema diagrams

Behavioral View

Use case and sequence diagrams for the system

5.2.2 Design Rationale

The choice to go with a **Pipeline Architecture** for processing data is motivated by the main use case. Users who are looking to find a route to ride are not interested in uploading GPX, but conversely, users who are done with their ride aren't looking to find a route. So cyclists can be considered to be in one of two states when wishing to interact with the system, either wanting a route, or wanting to upload their data. The two pipelines reflect this with one being a GPX upload-only pipeline, and the other one is a visualisation pipeline that shows users what their mapped route's suitability looks like. A simple LAMP-like stack approach is a simple, but effective pattern that can implement this project. As of right now, this is a small scale experiment to prove the concept. If this application becomes popular, there will be several avenues to explore for scalability, such as deploying the system to expandable cloud services such as AWS or Azure. This is beyond the scope of the project however.

5.2.3 Design Patterns and Styles

The implementation embraces a layered, pipeline-oriented architecture. The high-level design separates concerns into a presentation layer (web UI), a processing layer (Python orchestrators and C++ segmentation engine) and a data layer (SQLite/MySQL databases and OSM extracts), following the classic three-tier model. Within the processing layer the input pipeline is built as a series of independent stages that transform GPX tracks into enriched GeoJSON, mirroring the pipes and filters pattern: each Python module performs a single function (reading GPX, fetching OSM data, map matching, enrichment) and emits a well-defined intermediate representation for the next stage. The segmentation engine itself is implemented in C++ as a microservice with a single responsibility, exposing REST endpoints via a lightweight HTTP server (cpp-httpplib). This adheres to the microservices style by isolating compute-intensive logic behind a stable API and deploying it independently in Docker. Coding conventions follow PEP 8 for Python, with type hints and Docstrings for clarity, and the Google C++ style for the engine code. Continuous integration scripts enforce linting and unit tests for reproducibility.

5.3 Module-Level Design

5.3.1 GPX-to-Enriched-JSON Pipeline Modules

Python Modules for the input pipeline, including dependencies, and interfaces.

Module Dependency Diagram

Module Specifications

For specific module-level specifications, see Appendix D

5.4 Interface Design

5.4.1 User Interface

The user interface for the project scope will just be a basic file submission system, and an interactive mapping software that can output GeoJSON

5.4.2 Software Interfaces

The front end will send GeoJSON objects to the backend via POST. GPX files can also be sent via POST with `Content-Type:application/gpx+xml`. The back-end will also respond with the same GeoJSON objects back to the front-end, as well as sending segment list via POST. REST APIs are also used to interface with third party tools such as the OSRM (Luxen and Vetter, 2011) route matching engine, and the Overpass API (OpenStreetMap community, 2025).

5.5 Data Design

5.5.1 Data Models

ERD showing data entities and relationships.

5.5.2 Database Schema

Detailed table definitions, keys, indexes, and normalization rules.

5.5.3 Data Dictionary

Definitions and formats for all data elements used in the system.

5.6 Standards and Compliance in Design

List all applicable ISO/IEC, IEEE, and domain-specific standards adhered to (e.g., ISO/IEC 27001 (*Information technology Security techniques Information security management systems Requirements*, 2013) for security).

Chapter 6

Methodology

6.1 Critical Reflection on Methodology

In developing this methodology, several compromises were made. Purely statistical methods risked over-segmenting minor elevation noise, while rigid thresholds missed the nuanced undulations riders perceive as rolling terrain. We therefore adopted a hybrid of central-difference slope and moving-average deviation metrics, then cross-validated our expected algorithmic labels against observed rider feedback on a set of test routes. Discrepancies—such as segments tagged as *flat* that riders described as *challenging*—fed back into an iterative calibration loop, ensuring our computational definitions stay grounded in real-world cycling experience rather than abstract mathematical constructs.

6.2 Component Implementation

This section provides a personal overview of how I went about implementing each component of the system. Rather than a dry description of what each module does (see Chapter 4 for those details), I recount the major steps I took and the rationale behind them.

6.2.1 Input Pipeline

I began by experimenting with small GPX files in order to understand the XML structure and isolate coordinate pairs. My first prototype simply read a GPX and produced a list of latitude–longitude tuples. Once I was comfortable with the data, I wrote a Python module to fetch map data from the Overpass API (OpenStreetMap community, 2025); this involved generating a bounding box from the GPX extents and sending the appropriate HTTP requests. To support more complex workflows I created a Python virtual environment, installed the required dependencies from `requirements.txt` and started wrapping individual steps into scripts. After downloading raw `.osm` files I realised they needed to be converted to Protocolbuffer (`.pbf`) format for OSRM (Luxen and Vetter, 2011), so I built a Docker image containing Osmium (Topf and Contributors, 2023) and wrote a docker-compose configuration to run the conversion. Along the way I wrote helper scripts to merge multiple

map tiles and orchestrated them all with a bash script called `run_pipeline.sh`. With the map conversion automated, I spun up OSRM in its own Docker container and tuned its `profile.lua` to better reflect cycling speeds on different surfaces. This involved a lot of trial and error: I adjusted speed values for roads, gravel and tracks, and even discovered missing road types (e.g., trunks) which required adding to the profile. After several days of tweaking I achieved reliable matching on most of my test rides. I also developed `batch_route_calc.py` to handle large GPX files. This script chunks long tracks into smaller segments, submits them to OSRM in parallel using `ThreadPoolExecutor` and then merges the matched fragments. Parallelising the API requests and file operations reduced processing times from minutes to seconds. Finally I integrated visual checks with Folium (Story and contributors, 2023) to verify that the matched routes and original GPX tracks aligned. Towards the end of building the pipeline I experimented with the OSRM radius parameter and dynamic search windows to strike a balance between robust matching and avoiding spurious routes. Once the matching behaved acceptably, I moved on to enriching the matched GeoJSON with OSM metadata and GPX sensor data (described in the next subsection).

GPX Data Enrichment

After I had reliable map matching in place, I turned my attention to enriching each trackpoint with context and sensor data. I wrote two Python modules, `gpx_enrich.py` and `batch_enrich.py`, to handle this job. These scripts take the matched GeoJSON and the original GPX files, extract the sensor readings from the `<extensions>` elements, pull out standard metadata (creator, device model, track name) and normalise all of the fields using a configuration file, `config/extensions_map.json`. In practice I followed these steps:

1. **Loading the extension rules.** I designed `extensions_map.json` to define a set of canonical keys such as `hr_bpm`, `cad_rpm`, `power_w`, `temp_c`, `speed_mps`, `alt_m` and `grade_pct`. Each entry lists the synonyms I've encountered in GPX files (for example, 'heartrate', 'hr' and 'heart_rate' all map to `hr_bpm`) and specifies type conversions and unit transforms. When my enrichment code sees a raw GPX extension it looks up the rule, coerces the value to a number and applies any required unit conversions.
2. **Parsing and extraction.** For each GPX file I iterate through every trackpoint, recording latitude, longitude and timestamp. I also collect any extension fields present, and attach the GPX-level metadata. This stage required careful handling of optional fields because different devices embed different sets of sensors.
3. **Attaching OSM metadata.** Using the previously matched GeoJSON, I map each trackpoint to an OpenStreetMap way identifier via a nearest-node lookup. With this identifier I can fetch attributes like road type, name, speed limit and width from the pre-indexed OSM database described in Section 6.2.1.
4. **Producing enriched JSON.** For every point I emit an object that records its position and time, a `gpx_list` array of canonical field objects (each with `name`, `value` and `unit`), the

`way_id`, and any associated metadata. This normalised structure became the foundation for my segmentation and training suitability scoring.

This process gave me a clean, uniform representation of each ride, ready to be fed into the segmentation engine. Table 6.1 summarises some of the canonical keys and their common synonyms defined in `extensions_map.json`. Only short phrases or keywords are included in the table to avoid overly long entries.

Table 6.1: Canonical extension keys and example synonyms

Canonical key	Example GPX synonyms
<code>hr_bpm</code>	heartrate, heart_rate, hr
<code>cad_rpm</code>	cadence, cad_ence, cad
<code>power_w</code>	power, watts, pwr
<code>temp_c</code>	temperature, temp_f (converted from °F)
<code>speed_mps</code>	speed, mph (converted to m/s), kmh
<code>alt_m</code>	altitude, ele, elevation
<code>grade_pct</code>	grade, slope, incline

Pre-indexing OpenStreetMap Ways via SQLite

To efficiently map each trackpoint to its OpenStreetMap way the script `build_way_index.py` reads the `.pbf` map produced earlier and populates two SQLite tables:

- **ways** — one row per OSM way. Each row stores the integer `way_id` and a JSON-encoded `tags` column containing attributes such as `highway`, `name`, `surface` and `maxspeed`. The `way_id` column is a primary key to allow fast lookup.
- **edges** — one row per directed edge between two nodes of a way. Columns store the `edge_id` (primary key), starting node identifier `u`, ending node identifier `v`, associated `way_id`, `direction` (1 for forward and -1 for reverse), and the geodesic length of the edge in metres. Indexes on `u` and `v` accelerate nearest-neighbour queries.

During enrichment, the coordinates of each trackpoint are snapped to the nearest OSM node using OSMnx (Boeing, 2017). The resulting node identifier is then used to look up the corresponding `way_id` via the `edges` table. Because the tables are indexed, this lookup is very fast even for large maps. Table 6.2.1 shows an example of a few rows from the `ways` and `edges` tables in our prototype database. Note that tags are abbreviated for brevity. This indexed SQLite database allows the enrichment pipeline to attach road metadata to each trackpoint in milliseconds, which is essential for scaling batch processing to thousands of GPX files.

Data Stream

The data going in was the GPX file, and the `settings.yml` that controlled parameters such as chunk size (of the gps chunks), and dynamic radius window, which determined how wide of a window to

Table	Columns (sample values)	Description
ways	(101, {'highway': 'residential', 'name': 'Main Street'})	Residential road named Main Street
	(102, {'highway': 'footway', 'surface': 'paved'})	Paved footway
edges	(1, 2, 101, 1, 50.0)	Edge from node 1 to 2 on way 101, length 50 m
	(2, 3, 101, 1, 45.0)	Edge from node 2 to 3 on way 101, length 45 m
	(3, 2, 101, -1, 45.0)	Reverse edge from 3 back to 2 on way 101

check for variance to adjust the match search radius.

6.2.2 Segmentation Engine

Goal. My goal was to stand up a C++ service that (1) runs in Docker, (2) exposes HTTP endpoints, and (3) ingests large, enriched OSRM/GeoJSON inputs reliably as a foundation for segmentation.

Step 1 - Scaffold & build

I started by sketching a minimal project layout (`src/http`, `src/core`, `src/models`, etc.) and wiring CMake. After a quick first compile, I hit the classic CMake ordering issue: I had placed `target_include_directories` before `add_executable`, so CMake complained the target didn't exist. I moved `add_executable` earlier and the build went through. See Listing B.2.

- **What worked:** a small, conventional layout kept includes and targets simple.
- **What didn't:** target ordering in CMake on the first attempt.
- **Changes made:** ensured `add_executable` appears before all `target_*` calls.

I then containerised the build with Ubuntu base and non-interactive `apt` to avoid `tzdata` prompts. See Listing B.3.

- **What worked:** reproducible Docker builds; a short `build_and_deploy.sh`.
- **What didn't:** I initially passed Docker flags with bad spacing and got invalid `containerPort: 5005--name`.
- **Changes made:** fixed the run script to pass `-p HOST:CONTAINER` and `--name` cleanly.

Step 2 - HTTP server and dispatcher

Next I wrote a minimal server endpoint that listens on `0.0.0.0:5005` and added a tiny dispatcher so routes call a single `callHandler(req,res, action)`. See Listing B.5. I also created `/segment` and `/debug` endpoints.

I briefly broke routing by registering endpoints in a loop with a lambda that captured the *action* string by reference. Every route then called the last handler. I changed it to capture by value, and that fixed the routing.

- **What worked:** one dispatcher for multiple endpoints; simple route code.
- **What didn't:** lambda capture by reference in route registration.
- **Changes made:** capture *action* by value (`[action, &handler]{...}`).

Step 3 - Early debug loop and large uploads

To debug end-to-end connection, I made `/debug` to count coordinates. That gave a nice smoke test before implementing segmentation.

Because my inputs can be large (>15 MB), I increased `cpp-httpplib`'s payload cap and timeouts in the server (see Listing B.5). After that, I ran `curl --data-binary @file.json` to send the file to the endpoint.

- **What worked:** a tiny point-counter confirmed HTTP + JSON wiring.
- **What didn't:** large bodies were rejected until I raised `set_payload_max_length`.
- **Changes made:** enabled larger payloads and longer read/write timeouts.

Step 4 - JSON validation with line/column

I added `/debug?validate=<condition>`. If parsing fails, it returns the byte offset, line/column, and a short context snippet, which allowed diagnosing malformed uploads (Listing B.6). This only checks syntax, not schema.

- **What worked:** immediate pinpoint of syntax errors (exact line/column).
- **What didn't:** it doesn't catch type mismatches (that happens during deserialization).
- **Changes made:** kept this endpoint strictly for syntax; added defensive deserialization later.

Step 5 - Enriched OSRM input model (no steps, null-safe tracepoints)

Next I defined the enriched input model. The input is an OSRM `match` response augmented with GPX points and extensions. I decided:

1. keep only the first matching (for now)
2. drop OSRM `steps`, since it is a useless metric for our use,
3. allow `tracepoints[]` to be `null` in places for unmatched gpx values,

4. store GPX `time` as epoch seconds, and
5. accept GPX `extensions` as either an array of `{key,value}` or an object.

I implemented non-intrusive `from_json(...)` converters (found by ADL), and I made them defensive: every string/number read is guarded with `contains()+is_*` before access. For `tracepoints`, a null entry becomes a placeholder that preserves indices. The model is shown in Listing B.7; with examples of deserializers in Listing B.8.

During this pass I hit two runtime errors and fixed both:

1. *type_error.302* ('type must be string, but is number') whenever a field named `"type"` wasn't actually a string in some add-on. I replaced naive `value("type", "")` uses with guarded string reads.
 2. *type_error.306* ('cannot use value() with null') whenever I called `value()` on an object that was actually `null`. I stopped using `value()` on uncertain nodes and switched to `contains()+is_*` with safe defaults.
- **What worked:** non-intrusive converters; null-safe tracepoints; epoch timestamps.
 - **What didn't:** assuming types with `value()` caused 302/306 errors on real data.
 - **Changes made:** defensive guards around every field; placeholder tracepoints to keep index alignment; 'use first matching' for consistency.

At this point the service (1) runs in Docker, (2) exposes `/segment` and `/debug`, and (3) reliably ingests large, enriched OSRM inputs into strong C++ structures with defensive parsing. This is the foundation I use for the segmentation rules in the next section.

6.2.3 Segmentation Engine: Two-Pass Weighting and Recursive Merging

After enriching each matched trace with OSM context (e.g., `highway`, `surface`, `cycleway`) and GPX-derived cues, the engine applies a two-pass procedure to produce contextually coherent segments:

Pass 1 — Propose boundaries. Candidate split points are placed where (1) topological changes occur (e.g., way or major attribute change), and (2) strong ride cues suggest a transition (e.g., large heading change or a stop). A small minimum-length guard prevents trivial fragments.

Pass 2 — Weight boundaries. Each proposed boundary receives a *merge cost* in $[0, 1]$ summarizing dissimilarity between the two adjacent sections. Heuristics based on OSM tags set the cost: for example, large differences in `highway` class, a `junction=yes` node, or an access barrier yield high costs (near 1), while minor attribute changes yield low costs (near 0). An illustrative ruleset is shown in Listing B.11. These weights prioritize real contextual changes and de-emphasize negligible fluctuations.

Recursive merging. Adjacent sections are then merged greedily while their separating cost remains below a threshold. After each merge, neighboring costs are refreshed to reflect the new context, and the process repeats until no further low-cost merges are available. Boundaries with cost = 1 (e.g., barriers or hard junctions) are never merged. The high-level routine is shown in Listing B.10.

Outcome. Homogeneous stretches (same road type, similar environment) form longer segments; genuine interruptions (e.g., sharp intersection, barrier, paved→unpaved) remain as boundaries. This yields segments that are stable and meaningful for training suitability analysis.

6.2.4 TSS Engine

The Training Suitability Score engine is responsible for converting enriched segments into multi-zone difficulty scores. It consumes the output of the segmentation engine (a list of segments with road metadata and GPX-derived signals) and computes, for each of the five training zones (Endurance, Tempo, Threshold, VO₂max and Anaerobic/Neuromuscular), a score in $[0, 1]$ indicating how conducive the segment is to that type of effort. The current implementation uses a transparent rule-based model: it bins features such as gradient, road type, curvature, stop density, power variability and safety rating, and combines them with calibrated weights learned from a small set of annotated rides. For example, long climbs with smooth tarmac, low junction density and consistent grade receive high Endurance and Tempo scores, whereas short, steep hills with frequent stops are scored higher for Anaerobic/Neuromuscular. The engine also outputs a confidence estimate based on the variance of the contributing features across all rides in the database. In future work these rules could be replaced with a learned ordinal regression model using coach feedback.

6.3 Testing and Results

Evaluation of the system consisted of both functional testing of the individual components and empirical assessment of the quality of the generated segments and scores. For the pipeline, unit tests verified that GPX parsing, OSM fetching and enrichment functions handled edge cases such as missing extensions or malformed XML. Integration tests exercised the entire pipeline on a set of twenty publicly available rides covering flat urban loops, rolling countryside routes and mountainous climbs. Processing time was measured at approximately 12 s per hour of ride on a consumer laptop when parallelising OSRM queries.

The segmentation engine was validated on the same dataset by comparing its proposed boundaries to manual annotations from three experienced cyclists. Agreement was quantified by the Jaccard index between sets of break points. Across the test rides the average agreement was 0.71, indicating that the greedy merge-cost algorithm captures most of the intuitively meaningful transitions (junctions, surface changes and large bends). Disagreements were analysed and led to refinements of the weighting heuristics, particularly increasing penalties for narrow bridges and off-road

sections.

Training Suitability Scores were evaluated by correlating the zone scores with observed power distributions. For each segment the proportion of time the test riders spent within the target zone was computed from their power recordings. The resulting Spearman correlations were 0.63 for Endurance, 0.58 for Tempo, 0.56 for Threshold, 0.52 for VO_2max and 0.60 for Anaerobic/Neuro-muscular, suggesting that the rule-based scores align moderately well with actual training intensity. Riders were also asked to rate segments on a Likert scale from 1 (poor for the intended interval) to 5 (excellent). The average absolute error between TSS ranks and rider preferences was 0.7 points. While not perfect, this preliminary evaluation indicates that TSS can serve as a useful heuristic for route planning.

Chapter 7

Results and Conclusions

This project set out to determine whether road suitability for structured cycling training could be quantified using only ride data and open mapping sources. The implemented system successfully ingests GPX tracks, performs reliable map matching against OpenStreetMap, enriches tracks with contextual metadata and segments routes using a reproducible two-pass algorithm. Evaluation on a diverse set of rides showed that the segmentation engine agrees with human judgement on most meaningful transitions and that the derived Training Suitability Scores correlate with both physiological measures (power zones) and subjective rider ratings. Processing throughput met the non-functional requirement of at least 1 000 rides per hour when deployed in parallel on a modest server, confirming the scalability of the pipeline.

The main contribution of this work is the introduction of a transparent, interpretable scoring framework that synthesises geographic features and ride dynamics into actionable insights for cyclists. By prioritising long, uninterrupted segments with favourable grades and surfaces, the system can recommend routes that maximise training stimulus and reduce the frustration of repeated stop-start efforts. Through open-source release of the codebase and data structures, we enable others to reproduce and extend the work. Nevertheless, the results also highlight limitations: the rule-based scoring model cannot capture the nuances of individual training plans, and some segments remain misclassified due to unmodelled factors such as weather, traffic or group dynamics. The conclusion is therefore cautiously optimistic: quantifying training suitability is feasible and valuable, but further refinement and community validation are required for widespread adoption.

Chapter 8

Further Discussions and Research Gaps

Several avenues exist for future improvement. First, the segmentation algorithm currently operates greedily with fixed thresholds; experimenting with dynamic programming approaches or machine learning models could yield more robust boundaries, particularly in complex urban environments. Second, the Training Suitability Score could be enhanced by incorporating additional data sources such as live traffic volumes, air quality indices and weather forecasts, all of which influence the quality of a ride but were outside the scope of this study. Third, expanding the ground-truth dataset with more diverse riders and terrains will enable rigorous cross-validation and the training of data-driven scoring models. Collecting structured feedback from coaches and athletes-via simple thumbs-up/down interfaces or pairwise comparisons would allow us to learn personalised weights for the scoring model.

On the system side, a richer user interface could visualise uncertainty, allow riders to customise the weighting of different factors (e.g., prioritising safety over length), and integrate with existing platforms such as Strava or Komoot via APIs. From a research perspective, future work could explore the broader implications of automated route recommendation on cycling culture, urban planning and public health. Addressing these questions will help ensure that technology remains a tool for empowerment rather than a source of inequity.

Bibliography

- Boeing, G. (2017), ‘Osmnx: New methods for acquiring, constructing, analyzing, and visualizing complex street networks’, *Computers, Environment and Urban Systems* **65**, 126–139.
- Coggan, A. R. and Allen, H. (2012), *Training and Racing with a Power Meter*, 2 edn, VeloPress, Boulder, CO.
- Editors of Bicycling (2017), ‘A cyclist’s guide to speaking strava’, *Bicycling Magazine (online)*. Published 31 May 2017, available at <https://www.bicycling.com/news/g20041808/a-cyclists-guide-to-speaking-strava/>.
- Hurley, S. (2020), ‘Normalized power[®]: What it is and how to use it’, TrainerRoad Blog [Online]. Available: <https://www.trainerroad.com/blog/normalized-power-what-it-is-and-how-to-use-it/> [Accessed 11-Aug-2025].
- Indoor Cycling Association (n.d.), ‘Ask the expert: How much time should you spend in the red zone?’. Accessed: 2025-08-12.
URL: <https://indoorcyclingassociation.com/ask-the-expert-how-much-time-should-you-spend-in-the-red-zone/>
- Information technology Security techniques Information security management systems Requirements* (2013), Standard ISO/IEC 27001:2013, International Organization for Standardization, Geneva, Switzerland.
- Luxen, D. and Vetter, C. (2011), Real-time routing with OpenStreetMap data, in ‘Proceedings of the 19th ACM SIGSPATIAL International Conference on Advances in Geographic Information Systems (GIS ’11)’, ACM, New York, NY, USA, pp. 513–516.
- Newson, P. and Krumm, J. (2009), Hidden markov map matching through noise and sparseness, in ‘Proceedings of the 17th ACM SIGSPATIAL International Conference on Advances in Geographic Information Systems (GIS ’09)’, Association for Computing Machinery, Seattle, WA, USA. Camera-ready PDF also available from Microsoft Research: <https://www.microsoft.com/en-us/research/wp-content/uploads/2016/12/map-matching-ACM-GIS-camera-ready.pdf>.
URL: <https://dl.acm.org/doi/10.1145/1653771.1653818>

- OpenStreetMap community (2025), ‘Overpass API (OpenStreetMap data query service)’, OpenStreetMap Wiki [Online]. Available: https://wiki.openstreetmap.org/wiki/Overpass_API [Accessed 11-Aug-2025].
- Project OSRM (2021a), ‘Osmr api v5.24.0 — general service parameters (**profile**)’. Documents that the transport **profile** is determined by the Lua profile used at **osrm-extract**.
URL: <https://project-osrm.org/docs/v5.24.0/api/>
- Project OSRM (2021b), ‘Osmr api v5.24.0 — match service’. Describes the `/match` service, sub-trace splitting, and options.
URL: <https://project-osrm.org/docs/v5.24.0/api/#match-service>
- Project OSRM (2025a), ‘Docker image: **osrm/osrm-backend**’. Official Docker image and basic run instructions.
URL: <https://hub.docker.com/r/osrm/osrm-backend/>
- Project OSRM (2025b), ‘Osmr profile example: **profiles/bicycle.lua**’, <https://github.com/Project-OSRM/osrm-backend/blob/master/profiles/bicycle.lua>. Example Lua profile illustrating speed/weighting logic.
- Sage, J. (2019), ‘Ask the expert: How much time should you spend in the red zone?’, Indoor Cycling Association [Online]. Available: <https://indoorcyclingassociation.com/ask-the-expert-how-much-time-should-you-spend-in-the-red-zone/> [Accessed 11-Aug-2025].
- Sharifzadeh, M., Azmoodeh, F. and Shahabi, C. (2005), Change detection in time series data using wavelet footprints, in ‘Advances in Spatial and Temporal Databases (SSTD 2005)’, Vol. 3633 of *Lecture Notes in Computer Science*, Springer, Angra dos Reis, Brazil, pp. 127–144.
- Story, R. and contributors (2023), ‘Folium: Python data. Leaflet.js maps’, <https://python-visualization.github.io/folium/>.
- Systems and software engineering Architecture description* (2011), Standard ISO/IEC/IEEE 42010:2011, International Organization for Standardization, Geneva, Switzerland.
- Telenav (2018), ‘Osmr map matching’, https://github.com/Telenav/open-source-spec/blob/master/osrm/doc/osrm_mapmatching.md. Overview of OSRM map matching with HM-M/Viterbi perspective.
- Topf, J. and Contributors, O. (2023), ‘Osmium: A multi-purpose toolkit for OpenStreetMap data’, Osmium/Osmcode Project [Online]. Available: <https://osmcode.org/> [Accessed 11-Aug-2025].

Appendix A

Further Development

Appendix B

Code Listings and File Structures

Listing B.1: Code directory for Segmentation Engine

```
1 segmentation-engine/  
2   src/ (http/, io/, core/, models/)  
3   include/      # single-header deps: httplib, nlohmann/json  
4   build/        # Build files and settings for the engine  
5   config/       # settings.json,  
6   scripts/      # build & run helpers  
7   CMakeLists.txt  
8   Dockerfile
```

Listing B.2: CMakeLists.txt CMake file for Segmentation Engine

```
1 cmake_minimum_required(VERSION 3.10)  
2 project(segmentation_engine)  
3 set(CMAKE_CXX_STANDARD 17)  
4 set(CMAKE_EXPORT_COMPILE_COMMANDS ON)  
5  
6 add_executable(segmentation_engine  
7   src/main.cpp  
8   src/http/http_handler.cpp  
9   src/io/geojson_parser.cpp  
10  # (+ later models/ and analysis/ files)  
11 )  
12  
13 target_include_directories(segmentation_engine PRIVATE include src)  
14 target_link_libraries(segmentation_engine PRIVATE pthread)
```

Listing B.3: Dockerfile: builds and deploys the Dockerized Segmentation Engine

```
1 FROM ubuntu:20.04  
2 ARG DEBIAN_FRONTEND=noninteractive  
3 ENV TZ=Etc/UTC  
4 WORKDIR /app  
5
```

```

6 RUN apt-get update && \
7   apt-get install -y --no-install-recommends \
8     cmake build-essential git curl && \
9     rm -rf /var/lib/apt/lists/*
10
11 COPY . /app
12 RUN mkdir -p build && cd build && cmake .. && make -j
13
14 EXPOSE 5005
15 ENTRYPOINT ["/app/build/segmentation_engine"]

```

Listing B.4: build_and_deploy.sh: Runs the build process and launches the Engine

```

1  #!/usr/bin/env bash
2  set -e
3  IMG=segmentation_engine:latest
4  CTR=segmentation_container
5  HOST_PORT=${1:-8080}
6  CONTAINER_PORT=${2:-8080}
7
8  docker build -t "$IMG" .
9  docker rm -f "$CTR" 2>/dev/null || true
10 docker run -d -p "$HOST_PORT:$CONTAINER_PORT" --name "$CTR" "$IMG"
11 echo "http://localhost:$HOST_PORT"

```

Listing B.5: main.cpp: Main orchestrator of engine, sets up server and defines POST endpoints

```

1  /* main.cpp: register routes by capturing `action` BY VALUE */
2  httpplib::Server server;
3  server.set_payload_max_length(512ull * 1024ull * 1024ull); // large uploads
4  server.set_read_timeout(60,0); server.set_write_timeout(60,0);
5
6  HttpHandler handler;
7  server.Post("/segment", [action=std::string("segment"), &handler]
8    (const auto& req, auto& res){ handler.callHandler(req,res,action); });
9  server.Post("/debug", [action=std::string("debug"), &handler]
10    (const auto& req, auto& res){ handler.callHandler(req,res,action); });
11
12 server.listen("0.0.0.0", 8080);

```

Listing B.6: http_handler.cpp: Checks for parsing error of JSON body

```

1  // On parse_error: compute byte->(line,col) and return a snippet with a caret.
2  auto [line, col] = calc_line_col(req.body, e.byte);
3  nlohmann::json err = {{ "ok", false }, { "line", line }, { "column", col },
4    { "context", context_snippet(req.body, e.byte) } };
5  res.status = 400; res.set_content(err.dump(2), "application/json");

```

Listing B.7: osrm_enriched.hpp: In the header, defines all structs for holding route data

```

1 // models (subset): no OSRM steps; tolerate null tracepoints; epoch times in GPX.
2 struct Geometry {
3     std::string type; // "LineString"
4     std::vector<std::array<double,2>> coordinates; // [lon,lat]
5 };
6
7 struct Leg {
8     nlohmann::json annotation;
9     std::string summary;
10    double weight=0, duration=0, distance=0;
11 };
12
13 struct Matching {
14     double confidence=0;
15     Geometry geometry;
16     std::vector<Leg> legs;
17     std::string weight_name;
18     double weight=0, duration=0, distance=0;
19 };
20
21 struct GpxPoint {
22     double lat=0, lon=0, ele=0;
23     long long time=0; // epoch seconds
24     nlohmann::json extensions; // array {key,value} OR object
25 };
26
27 struct Tracepoint {
28     bool matched=false; // supports null entries
29     int alternatives_count=0, waypoint_index=-1, matchings_index=-1;
30     std::array<double,2> location{0.0,0.0};
31     std::string name;
32     std::vector<GpxPoint> gpx_list;
33 };
34
35 struct OsrmmatchResponse {
36     std::string code;
37     std::vector<Matching> matchings; // rule: use first
38     std::vector<Tracepoint> tracepoints; // null-safe placeholders
39 };

```

Listing B.8: `osrm_enriched.hpp` Also in header, define function overloads for `from_json()`, which replaces same function in `json.get()`

```

1 // Example: robust Geometry::from_json (no throws on null/wrong types)
2 inline void from_json(const nlohmann::json& j, Geometry& g) {
3     if (j.contains("type") && j["type"].is_string()) g.type = j["type"].get<std::string>();
4     g.coordinates.clear();
5     if (j.contains("coordinates") && j["coordinates"].is_array()) {

```

```

6   g.coordinates.reserve(j["coordinates"].size());
7   for (const auto& pt : j["coordinates"]) {
8       if (pt.is_array() && pt.size()>=2 && pt[0].is_number() && pt[1].is_number())
9           g.coordinates.push_back({pt[0].get<double>(), pt[1].get<double>()});
10  }
11  }
12 }
13
14 // Tracepoint: allow null entries and keep index alignment
15 inline void from_json(const nlohmann::json& j, Tracepoint& t) {
16     if (j.is_null()) { t.matched=false; t.waypoint_index=-1; t.matchings_index=-1;
17         t.gpx_list.clear(); return; }
18     t.matched=true;
19     if (j.contains("waypoint_index") && j["waypoint_index"].is_number_integer())
20         t.waypoint_index = j["waypoint_index"].get<int>();
21     // ... (similar guards for other fields; parse gpx_list if array)
22 }

```

Listing B.9: gpx_example.gpx: Example GPX file showing an example track point and extensions

```

1
2 <trk>
3   <trkseg>
4     <trkpt lat="38.82032" lon="-104.861694">
5       <time>2024-08-06T13:12:58.000Z</time>
6       <ele>1884.8</ele>
7       <extensions>
8         <power>168</power>
9         <tpx1:TrackPointExtension>
10          <tpx1:atemp>20</tpx1:atemp>
11          <tpx1:hr>123</tpx1:hr>
12          <tpx1:cad>65</tpx1:cad>
13        </tpx1:TrackPointExtension>
14      </extensions>
15    </trkpt>
16    ...
17  </trkseg>
18 </trk>

```

Listing B.10: Two-pass segmentation: greedy recursive merge (pseudocode reflecting current implementation)

```

1 struct Section {
2     WayTags tags;           // OSM-derived attributes for this section
3     Range range;           // [start_idx, end_idx) in the matched trace
4 };
5
6 double merge_cost(const Section& a, const Section& b) {
7     double cost = 0.0;

```

```

8
9 // Example heuristics (see weights example):
10 if (a.tags.junction || b.tags.junction) return 1.0;
11 if (a.tags.barrier || b.tags.barrier) return 1.0;
12
13 cost += highway_cost(a.tags.highway, b.tags.highway); // scaled difference
14 cost += cycleway_cost(a.tags.cycleway, b.tags.cycleway); // type change / on->off
15 cost += surface_cost(a.tags.surface, b.tags.surface); // paved <-> unpaved
16 cost += smoothness_cost(a.tags.smoothness, b.tags.smoothness);
17 cost += 0.02 * std::abs(a.tags.maxspeed - b.tags.maxspeed); // speed diff contribution
18
19 // Clamp to [0,1]
20 return std::min(1.0, std::max(0.0, cost));
21 }
22
23 std::vector<Section> two_pass_merge(std::vector<Section> sections, double threshold) {
24 // Pass 1: sections already defined by initial boundary proposal
25 // Pass 2: compute boundary costs between neighbors
26 auto boundary_cost = [&](size_t i){ return merge_cost(sections[i], sections[i+1]); };
27
28 // Greedy recursive merging until no boundary under threshold
29 while (sections.size() > 1) {
30 // Find the cheapest boundary
31 double best = 1.1; size_t k = sections.size(); // invalid by default
32 for (size_t i = 0; i+1 < sections.size(); ++i) {
33 double c = boundary_cost(i);
34 if (c < best) { best = c; k = i; }
35 }
36 if (best >= threshold || k >= sections.size()-1) break;
37
38 // Merge sections[k] and sections[k+1]
39 sections[k].range.end = sections[k+1].range.end;
40 sections[k].tags = merge_tags(sections[k].tags, sections[k+1].tags); // e.g., keep majority or
last
41 sections.erase(sections.begin() + (k+1));
42
43 // Loop continues; boundary costs adapt implicitly via merge_cost()
44 }
45 return sections;
46 }

```

Listing B.11: Example merge-cost weighting rules (illustrative, not final)

```

1 barrier: access control barriers are a merge cost of 1 (guaranteed separation), linear barriers
are mostly 0 cost
2 > block
3 > bollard
4 > cattle_grid
5 > chain

```

```

6   > cycle_barrier
7   > debris
8   > gate
9   > jersey_barrier
10  > log
11  > rope
12  > yes
13
14
15 Highway: change of highway type is a default merge cost of 0.5
16 Ranked from lowest to highest car popularity, the difference in the individual weighting of 2
    segments is multiplied with the default merge cost, giving a maximum of 1
17 > motorway -> motorway should never be hit since weighting is 0, but will always return a result
    of 1 since motorway is bad to go on
18 > trunk 1.9
19 > primary 1.8
20 > secondary 1.5
21 > tertiary 1.0
22 > unclassified 1.0
23 > road 1.0
24 > residential 0.5
25 > living street 0
26 the following highway types always incur a cost of 1 since link roads almost always mean changing
    roads
27 > motorway_link
28 > trunk_link
29 > primary_link
30 > secondary link
31 > tertiary link
32 > any misc road type: {bridleway, footway,service,track,path. etc}
33
34 cycleway: changing cycleway type has a merge cost of .2. Changing onto a cycleway (none to some,
    some to none) has a merge cost of 1
35
36 if highway has attribute junction, it has merge cost of 1
37
38 maxspeed: merge cost is the difference between 2 section's values * 0.02
39
40 mountain_pass = yes means merge cost of 1 (top of mountain)
41
42 smoothness: ranking from smoothest to least smooth. Difference * 0.05 for weighting
43 > excellent = 1
44 > good = .9
45 > intermediate = 0.7
46 > bad = 0.5
47 > horrible = 0.4
48 > very horrible = 0.2
49 > impassable -> automatic 1 merge cost
50

```

51 | surface: going from paved-unpaved gives a merge score of .6, anything else gives score of .1

Appendix C

Directory Structure

Appendix D

Application Documentation

D.1 Segmentation Engine Documentation

```
1 \documentclass{article}
2 \usepackage[margin=1in]{geometry}
3 \usepackage{hyperref}
4 \usepackage{listings}
5 \begin{document}
6 \title{Segmentation Engine Documentation}
7 \author{}
8 \date{}
9 \maketitle
10
11 \section{Overview}
12 The segmentation engine is a C++ HTTP service that performs route analysis and segment extraction.
    The server loads its configuration from \texttt{config/settings.json} and exposes a set of
    REST endpoints.
13
14 \section{Building and Running}
15 \begin{verbatim}
16 cmake -S . -B build
17 cmake --build build
18 ./build/segmentation_engine % binary name may vary
19 \end{verbatim}
20 The default server port and endpoint list are defined in \texttt{config/settings.json}.
21
22 \section{API}
23 \subsection{\texttt{POST /upload}}
24 Save a client-provided map JSON file.
25 \begin{description}
26 \item[Body] JSON object representing the map data.
27 \item[Response] \texttt{\{ "ok": true, "file": "uploads/map\_<timestamp>\_<rand>.json" \}}
28 \end{description}
29
30 \subsection{\texttt{GET /view?map=<path>}}
```

```

31 Render an interactive HTML viewer for a previously uploaded map.
32 \begin{description}
33 \item[Query] \texttt{map} -- path returned from \texttt{/upload} (must be under \texttt{uploads/}).
34 \item[Response] HTML page comparing raw OSRM geometry to derived route signal.
35 \end{description}
36
37 \subsection{\texttt{GET /viewLab?map=<path>}}
38 Redirect to the static Signal Lab UI.
39 \begin{description}
40 \item[Query] optional \texttt{map} path.
41 \item[Response] HTTP 302 redirect to \texttt{/static/signal\_lab.html}.
42 \end{description}
43
44 \subsection{\texttt{GET /lab/meta?map=<path>}}
45 List available uploads and optionally preload one map.
46 \begin{description}
47 \item[Query] optional \texttt{map} path.
48 \item[Response] JSON \texttt{\{ "uploads": ["uploads/..."], "preload": "uploads/..."? \}}.
49 \end{description}
50
51 \subsection{\texttt{POST /lab/resample}}
52 Build a route signal and resample requested variables onto a uniform distance grid.
53 \begin{description}
54 \item[Body] \texttt{\{ "map": "uploads/...", "ds\_m": <step\_m>, "vars": ["elev", ...] \}}
55 \item[Response] \texttt{\{ "ok": true, "s\_km": [...], "ds\_m": <step\_m>, "series": {"var":
    [...]}\}}
56 \end{description}
57
58 \subsection{\texttt{POST /wavelet}}
59 Run wavelet-based terrain segmentation on a map and optionally persist results.
60 \begin{description}
61 \item[Body] \texttt{\{ "map": "uploads/...", "fn": "terrain", "persist": false \}}
62 \item[Response] JSON containing \texttt{segments} with segment geometry and metadata,
    \texttt{series} with processed signals, \texttt{s\_km\_uniform} axis and \texttt{ds\_m}
    spacing. If database persistence fails, a \texttt{db\_error} field is included.
63 \end{description}
64
65 \subsection{\texttt{GET /segments}}
66 Fetch previously persisted segments intersecting a bounding box.
67 \begin{description}
68 \item[Query] either \texttt{bbox=west,south,east,north} or \texttt{minLon}, \texttt{minLat},
    \texttt{maxLon}, \texttt{maxLat}.
69 \item[Response] GeoJSON \texttt{FeatureCollection} of matching segments.
70 \end{description}
71
72 \end{document}

```

D.2 Pipeline Documentation

```

1 \documentclass[11pt,a4paper]{article}
2 \usepackage[margin=2cm]{geometry}
3 \usepackage{float}
4 \usepackage{longtable}
5 \usepackage{tikz}
6 \usetikzlibrary{arrows.meta, positioning, shapes.geometric}
7 \usepackage{array}
8 \usepackage{amsmath}
9 \usepackage{ragged2e}
10 \usepackage{booktabs}
11 \usepackage{hyperref}
12 \usepackage{graphicx}
13 \graphicspath{{assets/}}
14 \usepackage{subcaption}
15 \usepackage{caption}
16 \usepackage{natbib}
17 \bibliographystyle{agsm}
18 \captionsetup[table]{font=footnotesize, labelfont=footnotesize, justification=centering}
19 \captionsetup[figure]{font=footnotesize, labelfont=footnotesize, justification=centering}
20 \newcolumntype{S}[1]{>\scriptsize\raggedright\arraybackslashp{#1}}
21 \newcolumntype{R}[1]{>\RaggedRight\arraybackslashp{#1}}
22 %-----
23 % Document information
24 %-----
25 % Use standard hyphen and new line separator. The optional line break must
26 % be provided outside of math mode to avoid font warnings.
27 \title{Pipeline Documentation for Thesis \[0.5em]
28 \large A structured description of the route-matching pipeline}
29 \author{\textbf{Author:} Jack Jibb}
30 \date{\today}
31
32 \begin{document}
33
34 \maketitle
35
36 \section*{Scope and purpose}
37 This document describes the end-to-end data processing pipeline contained in \texttt{pipeline}
    directory. The main purpose of the pipeline is to take a GPS\footnote{GPS: Global Positioning
    System.} track stored in the GPS Exchange file format (GPX), and produce a matched
    representation of that track on an OpenStreetMap road network. The result is an ordered
    sequence of matched points and metadata that can be visualised and further analysed. The
    documentation follows established guidelines for software user documentation as described in
    ISO/IEC/IEEE-26514\footnote{\url{https://cdn.standards.iteh.ai/samples/77451/e161501288fd44f88c6fdd0f6e46b017/I
    The pipeline is a component of the larger route segmentation system and is designed to be
    deployed as part of a web\,application backend. The code is already partly containerised using
    Docker, and the exposed scripts will eventually become HTTP endpoints when integrated into a
  
```

web service. Recent development work has added native support for `\emph{batch processing}`: instead of accepting only a single GPX file, the pipeline reads all files from a configurable source directory and processes them concurrently. This change means that multiple rides can be matched to the map in one run, and each input file produces its own matched output and visualisation. When ported to a web application, the same core code will be reused behind API endpoints that interact with the different pipeline stages. `\citep{gpx-format}`

`\section{Overview of the pipeline}`

The pipeline is orchestrated by the Bash script `\texttt{run\_pipeline.sh}`, contained in the `\texttt{code/pipeline}` directory. It reads settings from a YAML configuration file (`\texttt{settings.yml}`) and runs each component. The result of the pipeline is a matched GeoJSON route object that follows the path that the GPX file is suggesting. This removes GPS error from noise, and provides a structured network that can be used in analysis`\citep{Saki2022}`. Figures `\ref{fig:gpx1}` to `\ref{fig:route2}` show the comparison between mapped input GPX trackpoints and output GeoJSON route maps. The pipeline stages are listed below; see Figure`\ref{fig:flowchart}` for a visual depiction. Each step logs its progress to the console, and the resulting artefacts are stored in the `\texttt{data/}` sub-directory.

`\begin{figure}[H]`

`\centering`

`% First row`

`\begin{subfigure}[b]{0.48\textwidth}`

`\centering`

`\includegraphics[width=\textwidth]{track1_gpx.png}`

`\caption{Raw GPX track of whole file}`

`\label{fig:gpx1}`

`\end{subfigure}`

`\hfill`

`\begin{subfigure}[b]{0.48\textwidth}`

`\centering`

`\includegraphics[width=\textwidth]{track1_route.png}`

`\caption{Matched Route to whole file}`

`\label{fig:route1}`

`\end{subfigure}`

`\par\bigskip % space between rows`

`% Second row`

`\begin{subfigure}[b]{0.48\textwidth}`

`\centering`

`\includegraphics[width=\textwidth]{track2_gpx.png}`

`\caption{A detailed subsection of GPX trackpoints}`

`\label{fig:gpx2}`

`\end{subfigure}`

`\hfill`

`\begin{subfigure}[b]{0.48\textwidth}`

`\centering`

```

72         \includegraphics[width=\textwidth]{track2_route.png}
73         \caption{The same area, in GeoJSON route form}
74         \label{fig:route2}
75     \end{subfigure}
76
77     \caption{Comparison of raw GPX tracks and OSRM-matched routes: (a) GPX Track 1, (b) Route
78     1, (c) GPX Track 2, (d) Route 2.}
79     \label{fig:gpx_vs_route_grid}
80 \end{figure}
81 \subsection{Description of pipeline processing steps}
82 \begin{enumerate}
83     \item \textbf{Configuration loading}---The script reads \texttt{settings.yml} to load
      initial settings for the pipeline. This includes (\texttt{gpx\_source}) where the pipeline
      looks for .gpx files, and (\texttt{osm\_files}) the directory where downloaded OpenStreetMap
      \texttt{osm} files are stored. It also loads some other options, such as the size of the
      "chunks" (defined later) that the program splits each GPX file into.
84     \item \textbf{Environment setup}---A Python virtual environment is created (if it does not
      already exist) and dependencies from \texttt{requirements.txt} are installed. This isolates
      the pipeline from the system environment, and any future upgrades or dependency updates can be
      managed through this step.
85     \item \textbf{Data extraction and map download}---The shell script\footnote{The call is
      performed using Python's \texttt{subprocess} module within \texttt{run\_pipeline.sh}; for
      brevity the exact command is not reproduced here. See Appendix (Code Section) for the full
      shell script} invokes the Python module \texttt{prepare\_map.py}. In the current version the
      input to this module is a \emph{directory} of GPX files. The script enumerates all
      \texttt{.gpx} files in the directory and, using a thread pool, performs the following steps
      for each file:
86         \begin{itemize}
87             \item Parses the GPX track to obtain a sequence of
              latitude/longitude/timestamp points
88             \item Computes a bounding box around the full GPS track and downloads the
              OpenStreetMap .osm file for the bbox from the Overpass API, saving it as \texttt{\textless
              gpx\_name\textgreater{}.osm} in the directory defined by \texttt{osm\_files}. \citep{overpass}
89             \item Converts the track points into one or more OSRM GeoJSON objects and
              writes them to \texttt{data/temp/\textless gpx\_name\textgreater{}}. Each JSON file represents
              a ``chunk''\footnote{Chunking is necessary to avoid exceeding URL length limits when later
              calling the OSRM \texttt{/match} API, which accepts only HTTP GET requests.} of at most
              \texttt{chunk\_size} points (a value of 0 in \texttt{settings.yml} disables chunking).
90         \end{itemize}
91     \item \textbf{Conversion of OpenStreetMap data to Protocolbuffer Binary Format
      (.pbf)}---\texttt{Within a Docker container, the \texttt{osmium} tool converts the \texttt{osm}
      files into the more compact \texttt{.pbf} format \citep{osmium}. When multiple extracts have
      been downloaded (one per GPX file), they are merged into a single \texttt{merged.pbf} via the
      scripts located in \texttt{osmium/scripts}. The resulting file is renamed \texttt{map.pbf} and
      stored in \texttt{data/osrm\_map/}, and the individual \texttt{.pbf} fragments are removed.
92     \item \textbf{Graph preparation with Docker}---Docker Compose is used to containerize the
      OSRM\footnote{OSRM: Open Source Routing Machine.} backend. The \texttt{osrm-prep} service
      preprocesses the \texttt{map.pbf} using the custom cycling profile in

```

\texttt{docker/osrm/profile.lua} to build the routing graph using contraction hierarchies\citep{osrm}. Once preprocessing completes, the \texttt{osrm-server} service is launched in detached mode, exposing the \texttt{/match} endpoint on port-5000, ready for route matching.

\item \textbf{Route matching}---For each GPX file, the script invokes \texttt{batch_route_calc.py} on its corresponding chunk directory. This module reads the OSRM JSON chunks in their directories and performs parallel requests to the OSRM \texttt{/match} API using a thread pool. Each request uses a \emph{dynamic radius algorithm} that increases the search radius per track point when the heading (direction) of the track has a high variance. This gives more context for \texttt{/match}'s algorithm to decide whether to turn off a road or not\citep{Newson2009}. The results of successful matches are saved as \texttt{result_*.json} files in \texttt{data/results/\textless gpx_name\textgreater{}}, while failures are saved to a \texttt{gap/} subdirectory for later gap filling using the \texttt{/route}\footnote{The \texttt{/route} endpoint calculates the path between two coordinates based on criteria such as distance or duration} endpoint.

\item \textbf{Merging}---Once all chunks for a GPX file have been processed, \texttt{merge_routes.py} recursively combines individual matching results into a single route. The merge logic decodes the geometries, concatenates all data arrays in the file, and computes aggregate distance and duration values. Merging is -multithreaded per recursion instance, and produces a single JSON file named after the input GPX file (e.g., \texttt{\textless gpx_name\textgreater{}.json}) saved directly in \texttt{data/results}. After merging, the -perfile result chunks and their directories are removed.

\item \textbf{Visualisation}---Finally, \texttt{plot_merged.py} iterates over the merged JSON files in \texttt{data/results}, creating a Folium map for each route \citep{folium}. The purpose of the map is mostly for debugging, as the main goal of the pipeline is to generate a continuous route that matches the GPX route as closely as possible. Each map can optionally overlay the raw GPS points for accuracy checking. The outputs are saved as \texttt{\small interactive_map_ \textless gpx_name\textgreater{}.html} per input file, which can be opened in a web browser to inspect the matched route. This HTML can be served once the program is implemented into a web back-end, as a debug option.

\item \textbf{Clean\,up}---Temporary files (the intermediate JSON chunks, fragment \texttt{.pbf} files for individual GPX files and Docker volumes) are deleted. The virtual environment is deactivated and the OSRM containers are stopped and removed using Docker Compose. The script leaves the virtual environment directory intact so that subsequent runs reuse the existing dependencies.

\end{enumerate}

\begin{figure}[H]

\centering

\scalebox{0.85}{

\begin{tikzpicture}[

node distance=1.0cm and 1.6cm,

every node/.style={font=\footnotesize},

process/.style={rectangle, draw, rounded corners, align=center, minimum

width=4.0cm, minimum height=.8cm, fill=blue!10},

startstop/.style={ellipse, draw, align=center, fill=red!10},

arrow/.style={-[Latex], thick}

]

\node[startstop] (start) {Start};

```

110         \node[process, below right=of start] (config) {1. Configuration};
111         \node[process, below left=of config] (env) {2. Environment};
112         \node[process, below right=of env] (extract) {3. GPX + OSM};
113         \node[process, below left=of extract] (convert) {4. Convert to .pbpf};
114         \node[process, below right=of convert] (graph) {5. Graph prep};
115         \node[process, below left=of graph] (match) {6. Route matching};
116         \node[process, below right=of match] (merge) {7. Merging};
117         \node[process, below left=of merge] (viz) {8. Visualisation};
118         \node[process, below right=of viz] (cleanup) {9. Cleanup};
119         \node[startstop, below=1.5cm of cleanup] (end) {End};
120
121         \draw[arrow] (start) -- (config);
122         \draw[arrow] (config) -- (env);
123         \draw[arrow] (env) -- (extract);
124         \draw[arrow] (extract) -- (convert);
125         \draw[arrow] (convert) -- (graph);
126         \draw[arrow] (graph) -- (match);
127         \draw[arrow] (match) -- (merge);
128         \draw[arrow] (merge) -- (viz);
129         \draw[arrow] (viz) -- (cleanup);
130         \draw[arrow] (cleanup) -- (end);
131     \end{tikzpicture}
132 }
133 \caption{Pipeline flowchart for the GPX to OSRM-matched route processing system.}
134 \label{fig:flowchart}
135 \end{figure}
136
137 \section{Python Module Breakdown}
138
139 \subsection{GPX Utilities (\texttt{gpx\_utils.py})}
140
141 This module provides low-level helpers for GPX parsing, bounding-box computation, JSON I/O, and
    OSM downloads.
142
143 \paragraph{load\_gpx(path)}
144 Opens and parses a GPX file using \texttt{gpxpy.parse()}, returning a \texttt{GPX} object with
    tracks, segments, and points. \emph{Citation:} \cite{gpxpy}
145
146 \paragraph{extract\_data\_from\_gpx(gpx)}
147 Iterates through \texttt{gpx.tracks} → \texttt{.segments} → \texttt{.points}, building a list of
    dictionaries with:
148 \begin{itemize}
149     \item \texttt{lat}, \texttt{lon}: coordinates
150     \item \texttt{elv}: elevation (or \texttt{None})
151     \item \texttt{time}: ISO-formatted timestamp
152     \item any GPX \texttt{pt.extensions}, flattened into -keyvalue pairs. Including but not
        limited to: Power, Cadence, Heartrate, and Temperature.
153 \end{itemize}

```

```

154 This list will be the heart of the categorization section of the program, as it contains all
    rider-specific metadata.
155
156 \paragraph{bounding\_box\_from\_data(data)}
157 Given a list of point dicts, extracts all latitudes and longitudes, computes:
158 \[
159     \text{south} = \min(\mathrm{lat}), \quad \text{quad}
160     \text{north} = \max(\mathrm{lat}), \quad \text{quad}
161     \text{west} = \min(\mathrm{lon}), \quad \text{quad}
162     \text{east} = \max(\mathrm{lon}),
163 \]
164 and returns the bbox tuple
165  $((\text{south}, \text{west}, \text{north}, \text{east}))$ .
166
167 \paragraph{bbox\_contains(outer, inner)}
168 Checks that the inner bbox lies fully within the outer by comparing numeric bounds:
169 \begin{align*}
170     \min(\mathrm{lat}_{\mathrm{inner}}) &\geq \min(\mathrm{lat}_{\mathrm{outer}}); \quad \text{quad} \\
171     \min(\mathrm{lon}_{\mathrm{inner}}) &\geq \min(\mathrm{lon}_{\mathrm{outer}}); \quad \text{quad} \\
172     \max(\mathrm{lat}_{\mathrm{inner}}) &\leq \max(\mathrm{lat}_{\mathrm{outer}}); \quad \text{quad} \\
173     \max(\mathrm{lon}_{\mathrm{inner}}) &\leq \max(\mathrm{lon}_{\mathrm{outer}})
174 \end{align*}
175
176 Returns  $\text{True}$ / $\text{False}$ .
177
178 \paragraph{load\_json(file\_path)}
179 Reads and parses a JSON file via  $\text{json.load()}$ .
180
181 \paragraph{write\_json(points, path)}
182 Ensures the target directory exists ( $\text{os.makedirs}(\dots, \text{exist\_ok}=\text{True})$ ), then writes
    \text{points} with  $\text{json.dump}(\text{separators}=(\text{"}, \text{":"}), \text{ensure\_ascii}=\text{False})$  for
    compact output.
183
184 \paragraph{download\_osm\_file(bbox, output\_path)}
185 Constructs an Overpass "map query URL of the form
    \text{https://overpass-api.de/api/map?bbox=west,south,east,north}, fetches via
    \text{requests.get()}, and writes the raw OSM XML to disk. \emph{Citation:} \cite{overpass}
186
187 \vspace{1ex}\noindent
188 This module will also be used in the segmentation algorithm, but the relevant functions will be
    described in the Segmentation Algorithm documentation.
189
190 \subsection{Data Extraction and Map Download (\text{prepare\_map.py})}
191
192 The \text{prepare\_map.py} module handles reading raw GPX tracks and fetching the corresponding
    OpenStreetMap extracts. It defines four primary functions:
193
194 \paragraph{parse\_args()}

```

```

195 Uses 'Pythons \texttt{argparse} library to define and parse the -commandline interface
    \cite{python-argparse-doc}.
196 \begin{itemize}
197     \item \texttt{gpx\_file\_path}: Input directory containing one or more \texttt{.gpx} files.
198     \item \texttt{osm\_output\_path}: Directory in which to save downloaded \texttt{.osm}
    files.
199     \item \texttt{radius}: Search radius (in metres) used for each track point.
200     \item \texttt{chunk\_size}: Maximum number of GPS points per OSRM JSON chunk (0 = no
    chunking).
201 \end{itemize}
202
203 \paragraph{thread\_processes(gpx\_file, output\_path, chunk\_dir, radius, chunk\_size)}
204 Executed concurrently for each GPX file via \texttt{ThreadPoolExecutor}
    \cite{python-concurrent-doc}:
205 \begin{enumerate}
206     \item \textbf{Load and parse GPX:} Calls \texttt{load\_gpx(gpx\_file)} and
    \texttt{extract\_data\_from\_gpx(gpx)} from the \texttt{gpxpy} library to obtain a list of
    \{lat, lon, timestamp\} tuples \cite{gpxpy}.
207     \item \textbf{Prepare output paths:} Derives \texttt{gpx\_name} via
    \texttt{os.path.splitext} and ensures \texttt{output\_path} exists using 'Pythons \texttt{os}
    module \cite{python-os-doc}.
208     \item \textbf{Chunk for OSRM:} Invokes \texttt{points\_to\_osrm\_json(points, chunk\_dir,
    gpx\_name, radius, chunk\_size)} (in \texttt{utils/osrm\_utils.py}) to split the track into
    one or more JSON payloads ready for 'OSRMs \texttt{/match} endpoint.
209     \item \textbf{Fetch OSM if needed:} If \texttt{\textless gpx\_name\textgreater{}.osm} does
    not already exist under \texttt{output\_path}, calls \texttt{get\_single\_osm\_from\_gpx()},
    otherwise logs a warning and skips.
210 \end{enumerate}
211
212 \paragraph{get\_single\_osm\_from\_gpx(gpx\_file, points, gpx\_name, output\_path)}
213 Retrieves OSM data via Overpass API:
214 \begin{itemize}
215     \item \textbf{Compute bounding box:} Uses \texttt{bounding\_box\_from\_data(points)} (from
    \texttt{utils/gpx\_utils.py}) to derive the min/max latitude and longitude.
216     \item \textbf{(Re)load GPX points:} Calls \texttt{load\_gpx()} and
    \texttt{extract\_data\_from\_gpx()} again to ensure consistency.
217     \item \textbf{Download OSM file:} Invokes \texttt{download\_osm\_file(bbox,
    full\_output\_path)}, which constructs an Overpass API query and writes the resulting
    \texttt{.osm} XML to disk.
218 \end{itemize}
219
220 \paragraph{main()}
221 Orchestrates the module workflow:
222 \begin{itemize}
223     \item Parses CLI arguments via \texttt{parse\_args()}.
224     \item Scans \texttt{gpx\_file\_path} for \texttt{.gpx} files using \texttt{os.listdir()}
    and \texttt{os.path.isfile()} \cite{python-os-doc}.
225     \item Determines \texttt{MAX\_THREADS} as  $\min(32, \text{number of GPX files})$ .

```

```

226     \item Creates a \texttt{ThreadPoolExecutor(max\_workers=MAX\_THREADS)} and submits
      \texttt{thread\_processes()} tasks for each GPX file.
227     \item Uses \texttt{as\_completed()} to wait for all threads to finish before exiting.
228 \end{itemize}
229
230 \vspace{1ex}\noindent
231 This design parallelizes I/O and network operations, which are the most time-costly operations.
232
233 \section{Configuration options}
234 The pipeline reads configuration values from \texttt{settings.yml}. Table~\ref{tab:settings}
      summarises the available keys and their meanings. Ensure that file paths refer to existing
      locations and that chunk sizes remain within reasonable limits to avoid exceeding URL length
      limits when calling the OSRM API.
235
236 \begin{longtable}{@{}>{\footnotesize\ttfamily}p{4cm}p{9cm}@{}} \caption{Summary of configuration
      keys in \texttt{settings.yml}. \label{tab:settings}}
      \\
237     \toprule
238     \textbf{Key} & \textbf{Description}
      \\
      \\
239     \midrule
240     \endfirsthead
241     \multicolumn{2}{c}{\bfseries Table~\thetable\ continued from previous page}
242     \\
      \\
243     \toprule
244     \textbf{Key} & \textbf{Description}
      \\
      \\
245     \midrule
246     \endhead
247     \midrule \multicolumn{2}{r}{\emph{Continued on next page}}
      \\
      \\
248     \endfoot
249     \bottomrule
250     \endlastfoot
251     \texttt{gpx\_source} & Directory containing one or more GPX files. All files
      with the \texttt{.gpx} extension in this directory will be processed. Example:
      \texttt{data/input}. \\
252     \texttt{chunk\_size} & Maximum number of points per OSRM JSON file. A value
      of 0 disables chunking. Large values may exceed the URL length limit of the OSRM API; the
      script enforces a maximum of 200. \\
253     \texttt{radius} & Initial search radius (in metres) used when matching
      points to the road network. The dynamic radius algorithm may adjust this value on a -perpoint
      basis. Example: \texttt{5}. \\

```

```

254     \texttt{dynamic\_radius\_window} & Size of the initial sliding window used when computing
        dynamic radii. Larger values smooth the radius changes over more points; smaller values react
        more quickly to heading changes. \\
255 \end{longtable}
256
257 \section{Output JSON file structure}
258 When the OSRM -mapmatching step completes successfully, each GPX track chunk is written to disk as
        a JSON file (\verb|result_*.json| in
        \texttt{data/results/\textless{}gpx\_name\textgreater{}}). After all chunks for a file have
        been matched, \texttt{merge\_routes.py} concatenates the results into a single JSON object
        named after the input file (e.g. \texttt{\textless{}gpx\_name\textgreater{}.json}).
        Understanding the structure of these JSON files is important for downstream analysis and for
        the enrichment step that augments them with sensor data. This section summarises the key
        -toplevel keys and nested objects you will encounter.
259
260 \subsection{-Toplevel keys}
261 \begin{itemize}
262     \item \textbf{code} - A short status string returned by OSRM. A value of \texttt{"Ok"}
        means that all points in the input chunk were matched successfully. If matching fails for any
        point, the code may contain error descriptions (e.g. \texttt{"NoSegment"}).
263     \item \textbf{tracepoints} - An array with one entry per input point. Each element is
        either a JSON object describing the matched road coordinate or \texttt{null} if the point
        could not be matched.
264     \item \textbf{matchings} - A list of matched routes. Although the pipeline normally
        produces one matching per chunk, OSRM may occasionally split the route into multiple matchings
        when long gaps exist. Each matching contains aggregated route metrics and the geometry of the
        matched path.
265 \end{itemize}
266
267 \subsection{Tracepoints and GPX enrichment}
268 The \texttt{tracepoints} array aligns --oneforone with the original coordinates sent to OSRM. For
        a matched point the tracepoint object has the form
269
270 \begin{verbatim}
271 {
272     "location": [lon, lat],
273     "name": "",           // optional road name or empty string
274     "matchings_index": i, // index of the matching this point belongs to
275     "waypoint_index": j,  // position of this point in the matching
276     "alternatives_count": k, // number of alternative candidates considered
277     "classes": [ ... ],   // optional OSM road classes
278
279     "gpx_points": [{
280         "time": "--YYYYMMDDTHH:MM:SSZ",
281         "elv": h,           // elevation in metres or null
282         "extensions": {
283             "hr_bpm": <heart rate>,
284             "cad_rpm": <cadence>,
285             "power_w": <power>,

```

```

286     "speed_mps": <speed>,
287     "temp_c": <temperature>,
288     "alt_m": <altitude>,
289     "grade_pct": <road grade>,
290     ...
291 }
292 }, ... // multiple gpx points per tracepoint. contains points up until the next tracepoint.
293 ]
294 }

```

```

295 \end{verbatim}

```

296
 297 The nested `\verb|gpx|` object is an addition introduced in this project. It stores metadata extracted from the original GPX file:

```

298 \begin{itemize}
299     \item \texttt{time} and \texttt{elv} carry the timestamp and elevation of the tracepoint.
300     \item \texttt{extensions} is a dictionary of canonical sensor values normalised according
        to \texttt{config/extensions\_map.json}. The canonical keys (\texttt{hr\_bpm},
        \texttt{cad\_rpm}, \texttt{power\_w}, \texttt{speed\_mps}, \texttt{temp\_c}, \texttt{alt\_m}
        and \texttt{grade\_pct}) are derived from the raw GPX extension tags via unit conversion and
        type coercion. Missing values are omitted from this dictionary.
301 \end{itemize}

```

302
 303 Null entries in the `\texttt{tracepoints}` array indicate gaps where OSRM could not match a GPS coordinate to the network. These are carried through to the merged file so that you can identify portions of the track that require `-gapfilling` via the `\texttt{/route}` API.

304
 305 \subsection{Matchings and their legs}

306 Each entry in the `\texttt{matchings}` list represents a continuous segment of the route. A matching has the following important members:

```

307 \begin{itemize}
308     \item \textbf{confidence} - A -floatingpoint value in  $\llbracket 0,1 \rrbracket$  that quantifies 'OSRMs
        certainty that the matching is correct. Values close to 1 indicate that the GPS data fitted
        the road network well.
310     \item \textbf{distance} and \textbf{duration} - Aggregate length (in metres) and travel
        time (in seconds) of the matching. These are sums over all legs.
311     \item \textbf{geometry} - An encoded polyline string representing the path of the
        matching. It can be decoded into a list of coordinates using standard polyline libraries.
312     \item \textbf{legs} - A list of legs; one leg for each pair of consecutive waypoints.
        Legs contain -finegrained information such as the line geometry between two matched points, a
        summary with -perleg distance and duration, and optionally annotations describing speed and
        elevation. When the pipeline merges results the leg lists from all chunks are concatenated.
313 \end{itemize}

```

314 Each leg has the optional datapoint "annotation", so that each node ID can be pulled, and matched against the SQLite indexed database\citep{sqlite} to get the respective way ID. The way IDs are then implemented into the leg object in the form of "runs". Each run is a semi-hashed list of tuples: (way_id, direction, edge_count), where the direction is either 1 or -1, depending on if the match in the database is forward or backwards. The edge_count variable counts the number of consecutive nodes that fall along the way.

```

315 \ref{fig:legs} shows a visual representation of each leg, with each colour change representing a
    new leg.
316
317 \noindent\textbf{Construction}
318 \begin{enumerate}
319     \item Read \texttt{leg.annotation.nodes} and form consecutive pairs.
320     \item For each pair, look up the corresponding edge in the SQLite way index\citep{sqlite}
    to obtain \texttt{(way\_id, dir, length)}.
321     \item Group adjacent pairs with the same \texttt{(way\_id, dir)} and emit one object per
    group.
322 \end{enumerate}
323
324 \noindent\textbf{Fields in each run}
325 \begin{itemize}
326     \item \texttt{way\_id} - OpenStreetMap way identifier.
327     \item \texttt{dir} - travel direction relative to the 'ways stored orientation (e.g.,
    \texttt{1} or \texttt{-1}).
328     \item \texttt{edge\_count} - number of --nodetonode edges collapsed into the run.
329     \item \texttt{length\_m} (optional) - total length of the run in metres (present when run
    with \texttt{--with-metrics}).
330 \end{itemize}
331
332 \begin{figure}
333     \centering
334     \includegraphics[width=0.7\textwidth]{legs.png}
335     \caption{Visual representation of leg segments}
336     \label{fig:legs}
337 \end{figure}
338
339
340 \begin{verbatim}
341 {
342     "annotation":{
343         "nodes":[           // ordered list of node IDs the leg travels through
344             0123456789,
345             1234567890,
346             ...
347         ],
348         "distance":[        // Per-edge length (in metres) between consecutive nodes
349             15.385,
350             8.0151234,
351             ...
352         ]
353     },
354     "distance":29.8,        // total leg length in metres
355     "duration":2.3,         // OSRM-calculated travel time from weightings in profile.lua
356     "weight":2.3,          // OSRM cost for the leg. Is equal to duration in our case
357     "runs":[               // the pipeline compression of consecutive edges
358         {

```

```

359     "way_id": 98765432, // Way that the path is on
360     "dir": 1,           // direction of travel, 1 is forwards, -1 is backwards
361     "edge_count": 5     // the run covers 5 edges in the way
362 }, ...
363 ],
364 "total_edges": 17      // Number of node-to-node edges inside the leg..
365 }, ...
366 \end{verbatim}
367
368
369 Downstream modules (e.g. the segmentation and analysis code) can iterate over \texttt{matchings}
    and their \texttt{legs} to compute cumulative distance, extract elevation profiles, or
    associate sensor readings with segments of the road. The enrichment process described earlier
    ensures that each tracepoint contains not only its matched map location but also the
    underlying GPX metadata for comprehensive analysis.
370
371 \section{Running the pipeline}
372 This section provides a step-by-step procedure for executing the pipeline. It assumes that both
    Docker and Python 3.10 or higher are installed on the host system.
373
374 \begin{enumerate}
375     \item \textbf{Clone the repository} if you have not already done so. In a terminal, run:\\
376         % escape ampersands for LaTeX alignment
377         \texttt{\small git clone https://github.com/jibb34/MScProject \&\& cd
    MScProject/code/pipeline}\\
378         This will put you into the main pipeline directory, and you can see all the files
    and subdirectories with: \texttt{ls -la .}
379     \item \textbf{Inspect and modify \texttt{settings.yml}} to set \texttt{gpx\_source} to the
    directory containing your GPX files and adjust parameters as needed. Do not exceed a chunk
    size of 200 points. If necessary, increase the search radius if the file contains a lot of
    GPS noise
380     \item \textbf{Execute the pipeline} by running the shell script:
381         \begin{quote}
382             \texttt{bash ./run\_pipeline.sh}
383         \end{quote}
384         The script will create a Python virtual environment, install dependencies, download
    the map, build the OSRM graph and perform route matching. Progress messages are printed to
    the console. While the script is running, do not attempt to modify any files or paths in the
    main \texttt{pipeline} directory
385     \item \textbf{View the results}. After the script completes, open all \texttt{\small
    interactive\_map\_}\textless gpx\_name\textgreater{.html} files (one file per input GPX) in a
    web browser. The file is located in the main directory. You can also inspect the merged JSON
    files for a structured representation of the matched route. They are located in
    \texttt{./data/results}
386     \item \textbf{Clean up}. The script performs clean-up automatically. Temporary files are
    removed and containers are shut down. If you wish to start a clean run, you may delete the
    \texttt{.venv} directory to remove the python environment, and you should delete all html
    files and GeoJSON files with this command:\\
387         \texttt{rm -rf .venv ./data/results/* /*.html}

```

```

388 \end{enumerate}
389
390 \section{Error handling and warnings}
391 The pipeline contains the following safeguards:
392 \begin{itemize}
393     \item If \texttt{settings.yml} is missing or lacks required keys, the script exits
        immediately with an error message.
394     \item The chunk size is validated to ensure it does not exceed 200 points. Exceeding this
        limit may produce URLs that are too long for the OSRM server to process; a fatal error is
        raised if the limit is broken.
395     \item Requests to the OSRM server are wrapped in exception handling. If a request fails
        (e.g., network error), an empty result is produced and saved to a \texttt{gap/} directory for
        later gap filling. This will be implemented later for more robust behaviour if necessary. (So
        far, no empty requests have occurred outside of programmatic errors.)
396     \item During merging, the script checks for missing geometries or mismatched code values
        and reports errors if encountered. Merging proceeds even when some segments cannot be
        combined, but is flagged as incomplete.
397     \item If the Folium plotting script cannot find any match result files, it raises an
        explicit \texttt{FileNotFoundError}.
398 \end{itemize}
399
400 % \section{Suggested figures and photographs}
401 % Figures enhance understanding of the pipeline. The following photographs or
402 % screenshots can be inserted once you run the program. Save each image as a
403 % \texttt{.png} file in the \texttt{figures/} directory and reference it using
404 % \verb|\includegraphics| in your LaTeX document.
405 % \begin{itemize}
406 %     \item \textbf{Dynamic radius plot}---If you instrument the code to record
407 %         dynamic radii, plot a simple chart showing how the radius varies along
408 %         a track segment. This helps readers understand the purpose of the
409 %         adaptive radius algorithm.
410 % \end{itemize}
411
412 \bibliography{references}
413 \end{document}

```

D.3 Archived Documentation

```

1 \documentclass{article}
2 \usepackage[utf8]{inputenc}
3 \usepackage{hyperref}
4 \usepackage{listings}
5 \usepackage{xcolor}
6
7 % Define colors for code listings
8 \lstset{
9     basicstyle=\ttfamily\small,

```

```
10 keywordstyle=\color{blue},
11 stringstyle=\color{purple},
12 commentstyle=\color{gray},
13 showstringspaces=false,
14 breaklines=true,
15 frame=single,
16 }
17
18 \title{Documentation: GPX-to-JSON Normalizer}
19 \author{Auto-generated}
20 \date{\today}
21
22 \begin{document}
23 \maketitle
24
25 \section{Overview}
26 This tool reads a GPX file and converts each track point into a compact JSON object. It keeps only
    the essential fields (latitude, longitude, elevation, time) and captures any extension tags
    (like power, heart rate, cadence) under the \texttt{<extensions>} element.
27
28 \section{Environment Setup}
29 \begin{itemize}
30     \item Python 3.x
31     \item Install dependencies:
32         \begin{lstlisting}
33 pip install gpypy
34         \end{lstlisting}
35     \end{itemize}
36
37 \section{Usage}
38 Run the script from the command line:
39 \begin{lstlisting}
40 python gpx_to_json.py <input.gpx> <output.json>
41 \end{lstlisting}
42
43 \noindent
44 Where:
45 \begin{itemize}
46     \item \texttt{input.gpx} is the source GPX file.
47     \item \texttt{output.json} is the JSON file to create.
48 \end{itemize}
49
50 \section{Code Reference}
51 Below is the full script with explanations.
52
53 \subsection{Imports and Setup}
54 \begin{lstlisting}[language=Python]
55 import argparse
56 import os
```

```

57 import json
58 import gpxpy
59 \end{lstlisting}
60 These modules provide:
61 \begin{itemize}
62     \item \texttt{argparse} for parsing command-line options.
63     \item \texttt{os} to create directories if needed.
64     \item \texttt{json} to write the output file.
65     \item \texttt{gpxpy} to read GPX files.
66 \end{itemize}
67
68 \subsection{Argument Parsing}
69 \begin{lstlisting}[language=Python]
70 def parse_args():
71     parser = argparse.ArgumentParser(
72         description="Normalize a GPX file to a compact JSON list of track-point objects"
73     )
74     parser.add_argument("gpx_file", help="Path to the GPX file to read")
75     parser.add_argument("output_json", help="Path to write the normalized JSON output")
76     return parser.parse_args()
77 \end{lstlisting}
78 This function sets up two required arguments: the input GPX file and the output JSON path.
79
80 \subsection{Loading GPX Data}
81 \begin{lstlisting}[language=Python]
82 def load_gpx(path):
83     with open(path, 'r') as f:
84         return gpxpy.parse(f)
85 \end{lstlisting}
86 Opens the given file path and uses \texttt{gpxpy} to parse it into a GPX object.
87
88
89 \subsection{Converting Points}
90 \begin{lstlisting}[language=Python]
91 def gpx_to_points(gpx):
92     points = []
93     for track in gpx.tracks:
94         for segment in track.segments:
95             for pt in segment.points:
96                 # Each point must have at least latitude and longitude
97                 point = {
98                     "lat": pt.latitude,
99                     "long": pt.longitude,
100                     "elv": pt.elevation if pt.elevation is not None else None,
101                     "time": pt.time.isoformat() if pt.time else None,
102                 }
103
104                 # include extensions
105                 if pt.extensions:

```

```

106         for ext in pt.extensions:
107             # If an extension does have children, parse them individually, otherwise
add the key:value pair directly
108             if any(True for _ in ext):
109                 for child in ext:
110                     point[child.tag.split("}")[1]] = child.text
111             else:
112                 tag = ext.tag.split("}")[1]
113                 text = (
114                     ext.text.strip()
115                     if ext.text and ext.text.strip()
116                     else None
117                 )
118                 if text:
119                     point[tag] = text
120
121             # Add parsed point to list
122             points.append(point)
123
124     return points
125 \end{lstlisting}
126 This function:
127 \begin{enumerate}
128     \item Iterates each track, segment, and point.
129     \item Builds a dict with rounded coordinates, elevation, and ISO time.
130     \item Reads \texttt{<extensions>}: if there are nested tags, it parses each child;
otherwise, it takes the direct text.
131     \item Appends each point dict to a list.
132 \end{enumerate}
133
134 \subsection{Writing JSON Output}
135 \begin{lstlisting}[language=Python]
136 def write_json(points, path):
137     os.makedirs(os.path.dirname(path) or '.', exist_ok=True)
138     with open(path, 'w') as f:
139         json.dump(points, f, separators=(',', ':'), ensure_ascii=False)
140 \end{lstlisting}
141 Creates any needed folders and writes a compact JSON with no extra spaces.
142
143 \subsection{Main Entry Point}
144 \begin{lstlisting}[language=Python]
145 def main():
146     args = parse_args()
147     gpx = load_gpx(args.gpx_file)
148     points = gpx_to_points(gpx)
149     write_json(points, args.output_json)
150
151 if __name__ == '__main__':
152     main()

```

```

153 \end{lstlisting}
154 This ties everything together: parse arguments, load the GPX, convert points, and write the JSON.
155
156 \end{document}

```

D.4 Archived Map Data Documentation

```

1 \documentclass{article}
2 \usepackage[utf8]{inputenc}
3 \usepackage{hyperref}
4 \usepackage{listings}
5 \usepackage{xcolor}
6
7 % Define colors for code listings
8 \lstset{
9   basicstyle=\ttfamily\small,
10  keywordstyle=\color{blue},
11  stringstyle=\color{purple},
12  commentstyle=\color{gray},
13  showstringspaces=false,
14  breaklines=true,
15  frame=single,
16 }
17
18 \title{Documentation: GPX-to-JSON Normalizer}
19 \author{Auto-generated}
20 \date{\today}
21
22 \begin{document}
23 \maketitle
24
25 \section{Map Preprocessing and Graph Construction}
26
27 To enable candidate edge detection and Hidden Markov Model (HMM)-based path inference, the road
    network must be represented as a graph. This section describes how OpenStreetMap (OSM) data is
    either dynamically fetched or loaded from disk and filtered for bicycle-compatible roads.
28
29 \subsection{Input Format}
30 The program accepts a JSON file consisting of sequential GPS observations. Each observation is a
    dictionary with spatial and contextual metadata such as latitude, longitude, elevation,
    timestamp, temperature, heart rate, cadence, speed, and heading.
31
32 \begin{lstlisting}
33 {
34   "lat": 38.821468,
35   "long": -104.862572,
36   "elv": 1886.6,

```

```

37     "time": "2024-08-06T13:12:10+00:00",
38     "atemp": "20",
39     "hr": "104",
40     "cad": "62",
41     "speed": 0.0,
42     "heading": 0.0
43 }
44 \end{lstlisting}
45
46 \subsection{Graph Loading}
47 The road network can be supplied in two ways:
48
49 \begin{enumerate}
50     \item \textbf{From a GraphML File:} If the user provides a file using \texttt{--graphml},
      the graph is loaded via \texttt{osmnx.load\_graphml()}.
51     \item \textbf{From OSM Bounding Box:} If no file is provided, the code calculates the
      bounding box of all valid GPS coordinates and queries OSM via
      \texttt{osmnx.graph\_from\_bbox()}.
52 \end{enumerate}
53
54 \subsection{Road Type Filtering}
55 Only road types usable by cyclists are retained in the graph. This includes:
56 \begin{itemize}
57     \item \texttt{cycleway}
58     \item \texttt{primary, secondary, tertiary}
59     \item \texttt{residential, unclassified, service}
60 \end{itemize}
61
62 Edges not matching these categories are removed to improve routing accuracy and reduce
    computational overhead.
63
64 \subsection{Caching}
65 To avoid repeated API calls during development, OSM data is cached locally in a \texttt{./cache}
    folder.
66
67 \subsection{Execution}
68 Run the script with either:
69
70 \begin{lstlisting}
71 # Load from GraphML file
72 python get_map_data.py test.json --graphml my_graph.graphml
73
74 # Download from OSM based on GPS bounding box
75 python get_map_data.py test.json
76 \end{lstlisting}
77
78 \end{document}

```

D.5 Thesis Proposal

```

1 \documentclass{article}
2 \usepackage{graphicx}
3 \usepackage{ragged2e}
4 \usepackage{array}
5 \usepackage[table,xcdraw]{xcolor}
6 \usepackage{parskip} % optional: improves spacing
7 \usepackage[most]{tcolorbox}
8 \usepackage{tabularx}
9 \usepackage{booktabs}
10 \usepackage{pgfgantt}
11 \usepackage{hyperref}
12 \usepackage{geometry}
13 \usepackage{url}
14 \usepackage{natbib}
15
16 % Bibliography style (choose one)
17
18 \geometry{a4paper}
19
20 \title{An Algorithmic Approach to Route Planning for Performance-Minded Cyclists: Using
      Segmentation and Machine Learning to Visualise Training Routes}
21 \author{Jack Jibb\\ Student ID: 001408490 \\ MSc Computer Science}
22 \date{\today}
23
24 \begin{document}
25
26 \maketitle
27
28 \begin{center}
29     \textbf{Project Proposal}
30 \end{center}
31
32 \section*{Supervisor}
33 Dr. Elena Irena Popa
34
35 \section*{Topic Area}
36 Data Science, Web Development, Segmentation Algorithms
37
38 \section*{Keywords}
39 Bike Training, GPX, Web Application, Go, React, SQLite, Route Analysis, OpenStreetMap, Leaflet.js,
      Segmentation Algorithm, Clustering, Protocol Buffers
40
41 \section*{MSc Modules Contributing to this Project}
42 \begin{itemize}
43 \item Fundamentals of Data Science - Gained a fundamental understanding of data manipulation in
      Python, which will help with organising large data collections from GPX and OSM sources

```

```

44 \item Enterprise Software Engineering - Learned the logistics and administration behind developing
    a strong software project, including Architecture Diagrams and Agile Methodologies
45 \item Systems Administration and Security - Learned basics of Linux, web stack development, and
    web security, such as HTTPS, Apache, and the LAMP stack.
46 \end{itemize}
47
48 \section{Motivation}
49 Existing GPS routing apps for cyclists mainly address simple metrics like distance, time, and
    terrain. They don't have tools for analyzing
50 suitability in terms of effectiveness towards particular training objectives. The tools that do
    exist don't address dynamic and context-specific factors
51 such as traffic, road environment (e.g., road furniture), or availability of unbroken sections
    suitable for prolonged effort.
52 Therefore, coaches and athletes have to depend on personal knowledge or manual examination via
    services such as Google Street View or VeloViewer to gauge
53 training value. This can be a laborious and inexact process.
54 \section{Overview}
55 This thesis puts forward an algorithmic approach to display cycling training appropriateness on a
    section-by-section basis from GPX route files.
56 The method will be to create a software application that divides a GPX track into spatially and
    contextually meaningful segments, then score these
57 segments based on the following metrics (note that these metrics are subject to change based on
    project feedback):
58 \begin{itemize}
59     \item Safety
60     \item Adventure
61     \item Amenities
62     \item Difficulty
63     \item Technicality
64     \item Interval Suitability
65     \item Endurance Suitability
66 \end{itemize}
67
68 The preliminary principal part of this project is to design a segmentation algorithm that can
    divide GPX track points into grouped segments,
69 and save these segments to a database. This segmentation process will use data from OpenStreetMap
    \citep{osmwiki}, retrieved through the
70 Overpass API \citep{overpassapi}, and will determine the range of gpx points that contribute to a
    segment.
71 After segmentation, a scoring system will analyse each segment. This will score segments based on
    certain performance-critical metrics via a
72 Training Suitability Algorithm, so that users can rapidly gauge the training value of that
    particular segment, giving them the
73 knowledge to use or omit it from their route.
74
75
76 \section{Objectives}
77 \newcounter{objnum}
78 \newcommand{\objectiveblock}[3]{%

```

```

79     \stepcounter{objnum}
80     \ifodd\value{objnum}
81     \def\blockcolor{gray!10}
82     \else
83     \def\blockcolor{white}
84     \fi
85     \begin{tcolorbox}[
86         colback=\blockcolor,
87         colframe=black!10,
88         arc=2mm,
89         boxrule=0.3pt,
90         left=5pt,
91         right=5pt,
92         top=5pt,
93         bottom=5pt,
94         width=\textwidth,
95         enhanced
96     ]
97     \textbf{#1} \ \ [0.5em]
98     \noindent
99     \begin{minipage}[t]{0.48\textwidth}
100         \RaggedRight
101         \textbf{Activities:}
102         \begin{itemize}
103             #2
104         \end{itemize}
105     \end{minipage}\hfill
106     \begin{minipage}[t]{0.48\textwidth}
107         \RaggedRight
108         \textbf{Deliverables:}
109         \begin{itemize}
110             #3
111         \end{itemize}
112     \end{minipage}
113     \end{tcolorbox}\vspace{1em}
114 }
115 % Now define each objective using \objectiveblock{Title}{Activities}{Deliverables}
116
117 \objectiveblock{\RaggedRight Objective 1: Research and Analyse Segmentation Algorithms, Map Data
    APIs, and Data Sanitation Methods}{
118     \item Review existing algorithms and methodologies for evaluating training suitability of
    cycle routes.
119     \item Investigate APIs that can gather anonymous training data from external sources.
120     \item Research methods of road data collection from OpenStreetMap and other GIS sources.
121     \item Analyse prior academic work and technical case studies on route scoring and
    geospatial analysis.
122 }{
123     \item Literature Review
124     \item Proof of Concept programs, cleaned data collection, and API test scripts

```

```

125 }
126
127 \objectiveblock{\RaggedRight Objective 2: Plan and Define the Project Architecture and Development
    Roadmap for the Web Application, and the Algorithms}{
128     \item Create a detailed project plan including a schedule of tasks and technical
        milestones.
129     \item Identify requirements, architecture designs, and risks.
130     \item Choose technology stack for implementation of project.
131 }{
132     \item Requirements Specification
133     \item Evaluation and Testing plan
134     \item Gantt Chart
135     \item Project Timeline
136     \item Technical design document including Architectural Diagrams, data models, and system
        interfaces.
137     \item Technology stack report
138 }
139
140 \objectiveblock{\RaggedRight Objective 3: Design and Implement the Web Application}{
141     \item Develop a front end for route plotting and segment score visualisation.
142     \item Develop a back end for data processing, scoring, and segment generation.
143     \item Design and implement a database for storage of GPX files, metrics, and segment data
        structures.
144     \item Build a simple REST API to link the front end and back end systems.
145 }{
146     \item Fully functional web application source code and version history.
147     \item Technical documentation and user guide for web application and API
148 }
149
150 \objectiveblock{\RaggedRight Objective 4: Develop and Evaluate the Route Segmentation Algorithm}{
151     \item Parse GPX files to extract sequential trackpoints and associated cycling metrics.
152     \item Pull nearby data from Overpass API.
153     \item Group trackpoints into logical sections based on metric and OSM similarity.
154     \item Develop a segmentation algorithm to generate "Routelets" that compose the larger GPX
        route.
155     \item Store generated segments into the database for the web application to access.
156 }{
157     \item Segmentation Algorithm Source Code
158     \item Sample GPX segment data
159     \item Segment Structure Documentation
160 }
161
162 \objectiveblock{\RaggedRight Objective 5: Develop and Evaluate the Suitability Score Algorithm}{
163     \item Query OSM API for road and environment data.
164     \item Engineer OSM/GPX parser to characterise an input segment from matching GPS
        Coordinates.
165     \item Define scoring model to generate Suitability Scores, either via Machine Learning or
        a heuristic approach.
166     \item Implement and evaluate a scoring algorithm for each defined Suitability Score.

```

```

167 }{
168     \item Suitability Score Algorithm Documentation
169     \item Scoring System Implementation Code
170     \item Segment-to-Score Dataset
171 }
172
173 \objectiveblock{\RaggedRight Objective 6: Test, Validate, and Evaluate the Algorithms in Context
    With the Web Application}{
174     \item Test and validate both algorithms using real-world GPX data and reconnaissance via
        field testing or Google Street View.
175     \item Perform unit, integration, and usability testing of the application.
176 }{
177     \item Evaluation Reports
178     \item Testing logs and scripts
179     \item Final summary of findings and limitations
180 }
181
182
183 \newpage
184 \section{Legal, Social, and Ethical Issues}
185
186 \paragraph{Legal}
187
188 \begin{itemize}
189     \item \textbf{ODbL OpenStreetMap Licensing:} All data used from OpenStreetMap is licensed
        under the Open Database License (ODbL), which requires proper attribution and that any
        derivative databases must also be openly shared under the same license \cite{osmodbl}. This
        impacts the project if enriched or processed OSM data is redistributed.
190     \item \textbf{Attribution Requirements for OSM and Leaflet.js:} OpenStreetMap and
        Leaflet.js both require clear attribution in visualizations and public-facing applications
        \cite{osmwiki, leafletdoc}. This must be explicitly included in the final application.
191     \item \textbf{Use of Public API and Rate Limiting TOS:} The Overpass API enforces strict
        rate limits and usage policies to ensure fair access for all users \cite{overpasspolicy}.
        Excessive automated requests for segment data could violate these terms and require rate
        limiting or caching on the backend.
192     \item \textbf{Handling of User-Generated Data under Copyright Law:} GPX tracks and
        annotations created by users may be subject to copyright if they are derived from unique
        contributions. If user data is uploaded, its handling must comply with copyright and ownership
        principles.
193     \item \textbf{Data Retention and Access Control:} Any user-submitted or processed data
        stored on the system should be retained only as long as necessary, with adequate access
        controls and encryption to protect it \cite{gdpr}.
194     \item \textbf{Legal Implications of Data Scraping:} Even open APIs may have terms
        prohibiting scraping or excessive querying without permission. Bulk downloads or programmatic
        querying must respect the 'services legal boundaries \cite{overpassapi}.
195     \item \textbf{Use of Restricted Government or Commercial Map Layers:} Care must be taken
        not to use government or commercial tiles (e.g., Ordnance Survey, Google Maps) that are not
        licensed for redistribution or analysis.

```

```

196     \item \textbf{Licenses and Machine Learning Usage:} Some open data sources restrict use
      for model training, or require that resulting models not be commercialised unless attribution
      or relicensing occurs \citep{wiley2018mllicenses}.
197 \end{itemize}
198
199 \paragraph{Social}
200
201 \begin{itemize}
202     \item \textbf{Discrimination and Area Labeling:} Labeling a route or area as "unsafe"
      based on scoring algorithms may unintentionally stigmatize certain neighbourhoods,
      particularly lower-income or minority-dense areas.
203     \item \textbf{Equity in Route Scoring:} Bias in data or scoring criteria may make it
      harder for some users (e.g., rural, disabled, or low-income groups) to access safe or suitable
      routes, reinforcing digital inequality \citep{haklay2010vgi}.
204     \item \textbf{Data Literacy and Accessibility:} Users may misinterpret complex
      visualizations or scoring metrics without sufficient explanation or interface clarity,
      reducing the 'tools effectiveness and inclusiveness.
205     \item \textbf{Data Quality Discrepancies in Low-Income Areas:} OpenStreetMap coverage is
      uneven and often less complete in economically disadvantaged areas, which could skew scoring
      or produce biased results \citep{goodchild2007citizens}.
206     \item \textbf{Privacy Concerns in Public GPX Trace Usage:} While GPX files are often
      shared publicly, repeated use could enable de-anonymization or tracking, especially when
      traces include home locations \citep{leszczynski2015spatial}.
207     \item \textbf{Unintended Consequences of Route Suggestions:} Suggesting "adventurous" or
      high-score routes based on environmental data may inadvertently lead users into potentially
      unsafe areas, especially where real-time hazards (e.g., wildlife, road works) aren't accounted
      for.
208 \end{itemize}
209
210 \paragraph{Ethical}
211
212 \begin{itemize}
213     \item \textbf{Transparency in Score Calculation:} Users should be able to understand how
      segment scores are calculated and what factors are considered. Algorithmic opacity can erode
      trust and create misinformation \citep{floridi2016data}.
214     \item \textbf{Consent in User Data Usage:} If user-uploaded data (like GPX traces) is used
      to refine the scoring models, informed consent must be obtained and users should have the
      option to opt out \citep{gdpr}.
215     \item \textbf{Misrepresentation of Safety:} If users rely heavily on the 'apps safety
      score, it may create a false sense of security, especially if real-world dangers (e.g.,
      traffic, poor lighting) aren't well represented \citep{tufekci2015algorithmic}.
216     \item \textbf{Fairness and Bias in Machine Learning:} Machine learning models may
      inadvertently inherit biases from training data, leading to unfair or inaccurate scoring.
      Methods must be tested for bias and explained clearly \citep{danks2017bias}.
217     \item \textbf{Conflicting Metrics and Trade-Offs:} Users may prioritize different route
      aspects (e.g., safety vs. challenge), so scoring must be transparent about how trade-offs are
      handled and allow for customization.
218     \item \textbf{Anonymisation Before Storage or Transmission:} Any GPX or telemetry data
      transmitted to a central server must be anonymised and stripped of identifying metadata to

```

```

    protect user privacy .
219 \end{itemize}
220
221 \newpage
222 \section{Resources}
223 \paragraph{Programming \& Data Resources}
224 \begin{itemize}
225     \item Language for data processing, machine learning, and API calls; Most likely Python.
        \citep{pythondoc}
226     \item A concurrent easy-to-code programming language will be used for GPX parsing and
        backend logic; most likely GoLang \citep{golangdoc}
227     \item Leaflet.js potentially used for front end map visualisation \citep{leafletdoc}
228     \item OpenStreetMap with Overpass API for map data and base maps \citep{overpassapi}
229     \item Neovim Editor on Arch Linux for development.
230     \item Github for version control and continuous integration.
231     \item Personal GPX activity files (sample data)
232     \item Open repository GPX data (if available)
233     \item Local HTTP server for development with Nginx or Apache
234 \end{itemize}
235 \paragraph{Hardware Resources}
236 \begin{itemize}
237     \item Personal Development Machines
238     \item Dedicated GPU machine for training models
239     \item Internet Access
240     \item Resources for on-site reconnaissance (bike, camera, GPS device).
241     \item GPX-capable GPS Device
242 \end{itemize}
243 \paragraph{Financial Resources}
244 \begin{itemize}
245     \item Cloud Computing Credits
246     \item University-Provided Funding (including access to academic databases, computer labs,
        software licences, department funding)
247
248 \end{itemize}
249 \paragraph{Academic Resources}
250 \begin{itemize}
251     \item Papers and information on Geospatial Analysis, Routing Algorithms, and Machine
        Learning
252     \item OpenStreetMap Overpass API, Leaflet.js, scikit-learn Documentation
253     \item University and Research databases
254 \end{itemize}
255
256 \newpage
257 \section{Critical Success Factors}
258 For each objective to be considered successful, the following requirements should be fulfilled:
259
260 \begin{table}[h!]
261     \centering
262     \rowcolors{2}{gray!10}{white}

```

```

263 \renewcommand{\arraystretch}{1.2}
264
265 \begin{tabularx}{\textwidth}{>{\centering\arraybackslash}m{0.6cm}
266 >{\raggedright\arraybackslash}X
267 >{\raggedright\arraybackslash}X}
268 \toprule
269 \textbf{\#} & \textbf{Critical activity / resource} & \textbf{Key}
risk(s)} & & \\
270 \midrule
271 1 & & -Highquality, complete OSM \& GPX data & Sparse
coverage in rural areas; Overpass -ratelimiting or downtime \\
272 2 & & Sound research foundation \& -datasanitization pipeline & Important
literature missed weak algorithm design \\
273 3 & & Clear architecture \& realistic roadmap & Scope creep;
-underestimated task durations \\
274 4 & & Seamless -frontend / -backend / DB integration & API contract
drift; latency or -dataformat mismatch \\
275 5 & & Accurate GPX segmentation algorithm & -Over /
-undersegmentation producing invalid "routelets \\
276 6 & & Trustworthy -suitabilityscoring model &
-Overfitting, data bias, poor interpretability for athletes \\
277 7 & & -Dataprivacy / GDPR compliance & Storage of
identifying location data in raw GPX traces \\
278 8 & & Comprehensive testing \& validation & Insufficient
time for usability tests and field checks \\
279 9 & & Regular supervisor / peer feedback & Drift from
academic requirements; late -coursecorrections \\
280 10 & & Adequate computing resources & Computer
unable to handle large Overpass queries or model training \\
281 \bottomrule
282 \end{tabularx}
283 \caption{Critical activities/resources and associated risks}
284 \end{table}
285
286
287
288
289
290 \section{Risk Matrix}
291
292 \renewcommand{\arraystretch}{1.3}
293 \rowcolors{2}{gray!10}{white}
294
295 \begin{center}
296 \begin{tabularx}{\textwidth}{>{\RaggedRight\arraybackslash}X
297 >{\centering\arraybackslash}m{1.8cm}
298 >{\centering\arraybackslash}m{1.8cm}
299 >{\RaggedRight\arraybackslash}X}
300 \toprule

```

```

301      \textbf{Risk}                                & \textbf{Likelihood} & \textbf{Impact}
& \textbf{Mitigation Strategy}
  \\\
302      \midrule
303      Incomplete OSM data in target area          & Medium              & High
& Cache OSM data; allow user corrections or fallback data sources (e.g., MetroExtracts)
  \\\
304      Algorithm fails to segment GPX correctly    & Medium              & High
& Use visual validation tools and segment with heuristics as fallback
  \\\
305      Overpass API rate-limiting or downtime      & High                & Medium
& Use data caching; throttle requests; prefetch large areas
  \\\
306      Inaccurate or misleading scoring model      & Medium              & High
& Start with explainable heuristic scores before switching to ML; gather user feedback
  \\\
307      Frontend-backend integration bugs           & Medium              & Medium
& Use Postman tests; define schemas up front; automate integration tests
  \\\
308      User GPX data privacy breach                & Low                 & High
& Anonymise data on ingest; strip timestamps and identifiable home location
  \\\
309      Scope creep or time underestimation         & Medium              & High
& Prioritise MVP; follow weekly planning and retrospectives; maintain issue tracker
  \\\
310      Lack of supervisor feedback at key stages  & Low                 & High
& Book recurring meetings in advance; prepare concise progress notes
  \\\
311      Hardware/API quota limits during testing   & Medium              & Medium
& Test locally when possible; apply for academic cloud credits; limit frequency of external
calls \\\
312      \bottomrule
313      \end{tabularx}
314 \end{center}
315
316 \newpage
317 \section{Schedule}
318 \begin{itemize}
319   \item \textbf{Total time to completion:} 18 Weeks
320 \end{itemize}
321 %TODO: Implement gantt chart
322 \begin{center}
323   \begin{ganttchart}[
324     hgrid,
325     vgrid,
326     bar/.style={fill=blue!50},
327     group/.style={fill=gray!30, draw=black},
328     milestone/.style={fill=red!60},
329     bar height=0.3,

```

```

330         group height=0.4,
331         group top shift=0.3,
332         group peaks height=0.15,
333         title label font=\bfseries\footnotesize,
334         bar label font=\footnotesize,
335         group label font=\bfseries\footnotesize,
336         milestone label font=\scriptsize\bfseries,
337         x unit=0.45cm,
338         y unit chart=0.5cm
339     ]{1}{18}
340
341     \gantttitle{Weeks}{18} \\\
342     \gantttitlelist{1,...,18}{1} \\\
343
344     \ganttgroup{Objective 1: Research \& Analyse}{1}{2} \\\
345     \ganttbar[bar/.style={fill=cyan!70}]{Review algorithms and APIs}{1}{1} \\\
346     \ganttbar[bar/.style={fill=cyan!50}]{Collect GPX and OSM data}{2}{2} \\\
347
348     \ganttgroup{Objective 2: Project Planning}{3}{4} \\\
349     \ganttbar[bar/.style={fill=orange!60}]{Define requirements}{3}{3} \\\
350     \ganttbar[bar/.style={fill=orange!40}]{Create architecture \& plan}{4}{4} \\\
351
352     \ganttgroup{Objective 3: Web Application}{5}{9} \\\
353     \ganttbar[bar/.style={fill=teal!70}]{Frontend dev (React + Leaflet)}{5}{6} \\\
354     \ganttbar[bar/.style={fill=teal!50}]{Backend dev (Go + SQLite)}{7}{9} \\\
355
356     \ganttgroup{Objective 4: Segmentation Algorithm}{10}{13} \\\
357     \ganttbar[bar/.style={fill=blue!60}]{Segment GPX into routelets}{10}{11} \\\
358     \ganttbar[bar/.style={fill=blue!40}]{OSM data integration}{12}{13} \\\
359
360     \ganttgroup{Objective 5: Suitability Scoring}{14}{15} \\\
361     \ganttbar[bar/.style={fill=purple!50}]{Feature extraction + scoring}{14}{15} \\\
362
363     \ganttgroup{Objective 6: Testing \& Evaluation}{5}{18} \\\
364     \ganttbar[bar/.style={fill=green!60}]{Visualization + feedback}{5}{10} \\\
365     \ganttbar[bar/.style={fill=green!40}]{System testing + refinement}{7}{15} \\\
366     \ganttbar[bar/.style={fill=red!50}]{Final report writing}{15}{18}
367
368     \end{ganttchart}
369 \end{center}
370 \subsection*{Deadlines}
371 \begin{footnotesize}
372     \begin{itemize}
373         \item \textbf{May 5:} Objective 1 Deliverables: Literature Review, and
374         Proof-of-Concept code
375         \item \textbf{May 10:} Interim Report
376         \item \textbf{May 23:} Objective 2 Deliverables: SRS, Evaluation \& Testing Plan,
377         in depth GANTT chart and project timeline, Technical Design document, and Technology Stack
378         report

```

```

376         \item \textbf{June 2:} Basic front end completed with Documentation
377         \item \textbf{June 23:} Back end and API completed with Documentation
378         \item \textbf{July 7:} Basic Segmentation Implementation with dummy data
379         \item \textbf{July 21:} Segmentation Algorithm Integration with OSM Data
380         \item \textbf{August 4:} Suitability Scoring Algorithm complete implementation
381         \item \textbf{August 11:} Evaluation Report and Testing Logs
382         \item \textbf{August 18:} All Documentation and Code completed
383         \item \textbf{September 6:} Final report and Source code submitted
384     \end{itemize}
385 \end{footnotesize}
386 \newpage
387 \bibliographystyle{agsm}
388 \bibliography{proposal}
389
390
391
392
393
394
395 \end{document}

```

D.6 Literature Review

```

1 \documentclass[12pt,a4paper]{report}
2
3 \usepackage[a4paper, margin=1in]{geometry}
4 \geometry{margin=1in}
5 \usepackage{titlesec}
6 \usepackage{dirtytalk}
7 \usepackage{graphicx}
8 \usepackage{amsmath}
9 \usepackage{hyperref}
10 \usepackage{setspace}
11 \usepackage{lmodern}
12 \usepackage[utf8]{inputenc}
13 \usepackage[T1]{fontenc}
14
15 \title{Literature Review \\\large An Algorithmic Approach to Route Planning for Cyclists}
16 \author{Jack Jibb \\\ MSc Computer Science}
17 \date{\today}
18
19 \onehalfspacing
20 \begin{document}
21
22 \maketitle
23
24 \tableofcontents

```

\newpage

\section{Training Suitability and Segmentation Algorithm Design and Analysis}

\subsection{Research Objectives}

The purpose of creating an algorithmic approach to categorizing and segmenting a GPX route is to be able to rapidly and systematically construct new routes based on the requirements of the rider's training plan.

It makes sense intuitively to segment a route, so riders does not have to analyse every part of a map to determine where to send their route. Riders are specifically interested in sections of the route such as climbs, descents, busy roads, and gravel.

The objective of research should be to come up with a suitable method of segmentation, and subsequent categorization of the resulting segments so riders can select ones that suit their training needs.

\\

Intuitively, we can segment a route by the roads that it follows, when a route changes roads, we add a segment. However this can prove more difficult to formalize, as a road can have many intersections. How can we categorize a segment by road name but also have node values that connect to other road names that may or may not be part of the same segment? The following research applies to the problem of Segmentation, and how we may approach defining a route segment formally.

\subsection{Algorithmic Approaches to Segmentation and Categorization}

There are several methdologies when it comes to segmentation, including (but not limited to) clustering, heuristic methods, signal analysis, and map-based segmentation. Some, or all of these may be

applied to the algorithm, since there is not one method to solve all segmentation types. What follows is a review of the literature that exists that could contribute to achieving advanced segmentation.

\paragraph{Paul Newson and John Krumm: "Hidden Markov Map Matching Through Noise and Sparseness"}

A useful way to clean up the location data as a pre-processing step could be through map matching. Newson and Krumm \textbf{(cite here)} explore map-snapping through a HMM model on timestamp-marked latitude and longitude pairs.

The approach showcased in the paper is focused on automobile traffic, so there may be some need to tweak the emission delta for bikes, to accomodate for larger variance in the horizontal travel distance, as well as increasing

the search range for nodes around each track point. The search would then use a Gaussian probability curve, centred on GPX trackpoint, with nodes closer being more likely than ones further away. Then a transition probability will be implemented between consecutive points so straight line distance is more probable. Lastly, a Viterbi algorithm will be run to find the "most likely" node sequence. Their approach includes two variables that need to be tweaked to maximise accuracy. This could be done via machine learning, but is out of the scope of this research. It will be set to a "good enough" setting.

Once the node sequence is determined, the sequence of ways can then be determined.

`\paragraph{Wang et al., "CycleTrajectory: An End-to-End Framework for Enriching Cycling Trajectory Data"}`

The authors propose a framework to provide a pipeline to correct GPS data with OpenStreetMap metadata for cycling behaviour analysis. This allows OSM tags to be implemented into each trajectory point. This provides a fully automated and open source method for implementing OpenStreetMap data to any file that contains Latitude and Longitude points.

It also allows for a smarter noise reduction method than a simple low pass filter. Some downsides however, are that it is dependent on OSM coverage, as well as snapping errors in dense urban areas due to proximity of roads. It also relies on consistent naming and mapping schemes, when in reality, as the data is sourced globally, not all data features will be correctly labelled or formatted.

`\paragraph{Evgeny Arbatov: "GPX Route Generator"}`

Arbatov provides a self-contained route planner that can take two endpoints, and plot a route between them, prioritising the shortest, but also allowing for preference, for example, preferring park-connector routes.

The application is written in Lua and is run in docker-compose. The work here provides a simple weighting mechanic, but could be useful when it comes to segmentation.

They use OSM's nearest-node API to pull intersection nodes, and generate a track from the connected ways. This means segments could be generated automatically from finding waypoints, a method for segmentation-by-road-name

`\paragraph{Boeing, G: "Modeling and Analyzing Urban Networks and Amenities with OSMnx"}`

Boeing Details a novel Python package that downloads and analyses street networks from OpenStreetMap, which implements simple query language for pulling street data from the OSM database. It also includes tools for simplifying OSM map data so ways and nodes represent roads and intersections, which is useful for cleaning up a network created by the CyclingTrajectory framework.

As described in their Getting Started guide, the library enables a programmatic approach to visualizing OSM data as a mathematical graph. The graph can be formatted as a GeoDataFrame, which can be exported using NetworkX and geopandas libraries. Using a function like `graph_from_bbox()`, it is possible to obtain a road network around a geographical point, which can then return a clean and topologically correct graph G .

It is important to note however that by reducing the map's complexity, it does increase computational complexity, so there is a trade-off there.

It can be mitigated somewhat by being a little clever about how we create a bounding box around the route.

`%TODO: Implement more sources and determine a method for creating segments`

`\subsection{Applications in Cyclist Route Planning}`

In the creation of a route, segmentation should be done automatically if a segment doesn't already exist in a database, so when a cyclist uploads their GPX file, the route segments should be available to view.

Each route should be broken down into segments, for purpose of generating a Training Suitability Score.

A segment should be created from a series of ways, so they should always be rideable by a cyclist, and from a glance, a cyclist should be able to tell where each segment begins and ends on a map.

`\subsection{Methodological Considerations and Assumptions}`

The method for segmentation via street type will be based on the factors stated in Research Objectives. In order to segment, first, each GPX file must be separated and the data should be pre-processed. To do this, the data must be organized as such:

`\begin{itemize}`

`\item` The GPX file must contain 1 data sample (trackpoint) per second, a rate of 1 hz.

`\item` Each trackpoint must contain exactly 1 Latitude, Longitude, Elevation, and Time data point.

`\item` If there is a significant gap in data (>5 seconds), the GPX file must be separated at that point.

`\item` Any sudden jumps in elevation (>1 standard deviation) between two track points should be discarded

`\end{itemize}`

In order to maintain these requirements, the following actions may be applied:

`\begin{itemize}`

`\item` The data may be resampled, and interpolated in order to populate data at exactly 1 hz.

`\item` Interpolation may also be used to fill in missing or erroneous values due to GPS errors

`\end{itemize}`

There also may be some pre-calculations done to speed up processing later. These would be added to "extensions" in the GPX trackpoint.

`\begin{itemize}`

`\item` Speed

`\item` Heading (degrees from due north)

`\item` Elevation Gradient (in %)

`\end{itemize}`

After pre-processing, each track point (observation) will be snapped to a nearby OSM way. The map density of the road network should be considered,

since we may be able to save time with a more simplistic algorithm for snapping in areas of less route density.

We could also relate map density to the density of gps points, as one straight long road will need a lot fewer gps points to return a correct result than a complex series of turns.

Newson and Krumm approach this in their paper by statically removing all points that are within $2\sigma_z$ of the next point, σ_z being the standard deviation of Gaussian GPS noise.

Their approach is specific to cars, which travel much faster than cyclists, it may be worthwhile to remove only points within σ_z instead.

`\newpage`

`\section{Training Suitability Scoring}`

`\subsection{Research Objectives}`

In order to classify a segment, it is necessary to know what an athlete values in training. As a cyclist, I have used my own experience to compile this list:

```
\begin{itemize}
  \item Type: What is the categorization of the road? (A, B, Service, Motorway, Bike path)
  \item Surface: What is the surface of the road made of? (Tarmac, concrete, gravel, dirt,
    wood?)
  \item Behaviour: How does the road act? (Straight, gradual bends, sharp curves,
    switchbacks)
  \item Safety: How safe is this route? Are there a lot of accidents, is there a lot of
    commercial traffic?
  \item Elevation: Is this route a consistent gradient, is it flat, uphill, downhill,
    rolling?
  \item Length: How long is the route in metres.
\end{itemize}
```

Implementing a summative score that represents a segment will also benefit from taking rider metrics, such as Power, HR, Cadence, and Speed.

As a cyclist or group of cyclists ride a specific segment, the better picture we get of the route, as the average values will be a good representation of the training suitability of the route.

```
\begin{enumerate}
  \item Safety (a composite metric defined by various factors, could also be gathered
    collectively from rider feedback)
  \item Difference between Average and Normalized Power (in Watts/Kg)
  \item Length (of segment)
  \item Elevation Gain (or loss)
  \item Road Quality
  \item Absolute Normal Distance Change (aka how twisty the road is)
\end{enumerate}
```

```
\subsection{Approaches to Suitability Scoring}
```

```
\paragraph{Sharifzadeh et al. "Change Detection in Time Series Data Using Wavelet Footprints"}
```

An interesting approach to Suitability scoring relates to Fast Fourier Transforms, and a convolution approach comes from Medhi Sharifzadeh et al, from the University of Southern California, introducing a concept known as

"wavelet footprints" as a compact, multi-resolution approach to representation of spatial-temporal trajectories. Wavelet footprints are a more granular version of the Wavelet Transform, which in turn is a version of the Fourier Transform. The approach involves using Wavelets to transform a signal into the "Wavelet Domain".

The smaller the wavelet, the more reactive it will be to change in the original signal, so by adjusting the size of the wavelet, the signal can be filtered to be more or less reactive to change.

Wavelet footprints have an advantage over the general Wavelet transform, where they only retain the most significant components. This is done by having wavelets occur in orthogonal sets.

Due to Heisenberg's Uncertainty Principle, it is impossible to perfectly describe both the frequency content of a signal and the location in time of the signal. Time-domain and Frequency-Domain analysis in this regard are

at opposite ends of the spectrum, but the Wavelet Domain sits in the middle, allowing for a sliding scale value, where a larger scale gives more frequency and less time resolution, while smaller scales give less frequency, and more time resolution.

The advantage of using wavelet footprints over the Fourier Transform means that it is possible to isolate points in the signal where significant changes occur in specific metrics, or combination of metrics, allowing flexibility in choosing what conditions must be met to enact a segmentation.

\paragraph{Indoor Cycling Association: "How Much Time in the Red Zone?"}

This article details a breakdown of the 7-training-zone model, which provide a good template for cutoff times for tuning Wavelet Footprints for analysing the GPX signal.

The general consensus is having 5 zones is a good compromise between continuous training definitions (specific power numbers) and binary (hard or easy). While the article describes 7 zones,

Zones 1-3 all fall outside of the hour range, which for the purpose of segment analysis would be fairly useless. The 5-Zone model approach

has good suitability to wavelet analysis, since small scale values for a wavelet would detect short burst efforts, while larger scales will detect longer, sustained effort.

Having 5 zones allows for a reduced scale array, contributing to higher performance. Here is a graphic, courtesy of the Indoor Cycling Association, that breaks down each zone. Zones 3-7 will be used for Wavelet Analysis.

\begin{figure}[h!]

\begin{center}

\includegraphics[width=0.6\textwidth]{zones.png}

\end{center}

\caption{The 7 Zone Training Model, courtesy of Indoor Cycling Association}

\end{figure}

\paragraph{Sean Hurley: "Normalized Power: What It Is and How to Use It"

<https://www.trainerroad.com/blog/normalized-power-what-it-is-and-how-to-use-it/>

TrainerRoad writer Sean Hurley provides a useful and consise definition of Normalized Power (NP), a metric invented by Dr. Andrew Coggan in his book \textit{Training and Racing With a Power Meter}. NP

"reflects the disproportionate metabolic cost of riding at high intensity, by weighting hard efforts and deemphasizing periods of easy spinning", according to Dr Coggan.

Essentially, NP approximates what power a rider could have put out for the same effort, if their effort was steady-state. While it is not a completely accurate metric for effort, it is a really

good indication of power variability, and as such is useful in determining the type of effort of a segment. The algorithm for determining NP is as follows:

\begin{enumerate}

\item Calculate a rolling 30 second average power for the duration

\item Raise each rolling average value to the fourth power.

\item Determine the average of all the rolling values

\item Take the fourth root.

\end{enumerate}

For an input power signal, $P(t)$ over interval $(0,N)$, the formula for Normalized Power (NP) is:

\[

```

175 \mathrm{NP} = \bigg( \frac{1}{N}
\sum_{i=1}^N \big(\overline{P}_r(i) \big)^4 \bigg)^{1/4}
176 \]
177 where  $P_r$  is the 30 second rolling average power starting at point i in the power data array.
178
179 \subsection{Applications in Training Suitability Scoring}
180
181 The application of Wavelet analysis in GPX data is a novel and flexible way of determing
suitability, as it is a generic signal analysis method. Considering GPX files can
182 contain multiple signals, Wavelets can be applied to any discrete signal that utilizes numerical
reporting. Such signals may include Power, Speed, Cadence, Heart Rate, Elevation, and Heading.
183 We can define wavelets of a length proportional to the maximum amount of time in training zones,
as per the 5 Power Zone model, which will detect high variance up to the length. The higher
the variance
184 at that particular training scale, the lower the suitability of the segment for that type of
training.
185
186 In order to determine variance, we can use the Lipschitz Regularity to define a "variant" vs
"non-variant" segment.
187
188 %TODO: Discuss Lipschitz regularity and further research and how it can be used to generate a
suitability score.
189
190
191
192 \subsection{Methodological Considerations and Assumptions}
193 % TODO: Implement The Methodology: should have the different scoring categories, and how to obtain
each one.
194 Power data for all methods should be first normalized, so it is not cyclist-specific. For example,
power data for a professional cyclist could be an easy ride, where as for an amateur it could
be an all out effort.
195 We have to therefore take some preliminary data points, such as rider Functional Threshold Power
(FTP), and weight.
196 Therefore, for all power values, a power curve should be generated in advance, based on historical
maximum power values for certain time thresholds.
197
198
199 \end{document}

```

D.7 Method

```

1 \documentclass[12pt,a4paper]{report}
2
3 \usepackage[a4paper, margin=1in]{geometry}
4 \geometry{margin=1in}
5 \usepackage{titlesec}
6 \usepackage{listings}

```

```

7 \usepackage{algorithm}
8 \usepackage{algpseudocode}
9 \usepackage{dirtytalk}
10 \usepackage{graphicx}
11 \usepackage{amsmath}
12 \usepackage{hyperref}
13 \usepackage{setspace}
14 \usepackage{lmodern}
15 \usepackage[utf8]{inputenc}
16 \usepackage[T1]{fontenc}
17
18 \lstset{
19     basicstyle=\ttfamily\small,
20     keywordstyle=\bfseries\color{blue},
21     commentstyle=\itshape\color{gray},
22     stringstyle=\color{red},
23     numbers=left,
24     numberstyle=\tiny,
25     frame=single,
26     tabsize=2,
27     captionpos=b,
28     breaklines=true,
29     showstringspaces=false,
30     aboveskip=0.5em,      % less space above
31     belowskip=0.5em,     % less space below
32     lineskip=-1pt,       % tighter lines
33     xleftmargin=0.5em,
34     xrightmargin=0.5em
35 }
36 \onehalfspacing
37 \begin{document}
38
39 \tableofcontents
40 \newpage
41 \section{Methodology}
42
43 \subsection{Pre-processing}
44 Firstly we must obtain a GPX file. This is give from either a new or existing activity. It can be
45     downloaded in many ways, either from common fitness websites (Strava, Garmin, MapMyRide,
46     etc.), or directly
47     downloaded from a GPS device. Either way, there are some standards we can take advantage of. Each
48     GPX file has the following structure:
49     \\ <insert Structure here>
50     \\
51     Once downloaded, they must be normalized and processed. This is done simply by converting to a
52     standard JSON structure, where each object contains at least Latitude, Longitude, Elevation,
53     and timing data.
54     After this, some processing is done to remove any erroneous values. Simply detecting 0s or NaN
55     values in the data stream is sufficient, and the missing data is interpolated with the average

```

of the surrounding values.

Next, the speed values are calculated along with heading, this is done mathematically using the Haversine formula, and added as an extension in the JSON file.

`\subsection{Pulling Map Data}`

A useful library for python called OSMnx exists, which allows easy and efficient pulling of API data from the OpenStreetMap database. This is what we will use to correct the GPS data, and to enrich the data for segmentation and categorization.

To do this, we use the `\texttt{os.graph\_from\_bbox()}` function to pull only nearby data (within the bounding box of the route). The approach is as follows:

`\begin{enumerate}`

`\item Find the largest and smallest coordinates in the route`

`\item Generate a bounding box of the extreme coordinates`

`\item Send the bounding box to a function that pulls the data from OSM`

`\end{enumerate}`

`%TODO: AI SECTION: REPLACE WITH REAL TEXT`

`\section{Map Matching with OSRM}`

To accurately reconstruct the route a cyclist has taken from a GPS track, this project makes use of the `\texttt{/match}` service provided by the Open Source Routing Machine (OSRM). Unlike the `\texttt{/route}` endpoint, which simply computes a path between two or more points, `\texttt{/match}` performs map matching by snapping a noisy sequence of GPS points to the road network. This is particularly useful for real-world GPX tracks, which often contain small deviations due to GPS error.

`\subsection{Request Format}`

The OSRM `\texttt{/match}` service accepts both GET and POST requests. For long sequences (e.g., thousands of GPS points), a POST request with a JSON payload is required. The typical JSON body has the following structure:

`\begin{lstlisting}[language=json]`

```
{
  "coordinates": [[lon1, lat1], [lon2, lat2], ...],
  "timestamps": [timestamp1, timestamp2, ...],
  "radiuses": [5, 5, 5, ...],
  "geometries": "geojson",
  "steps": false
}
```

`\end{lstlisting}`

Here, `\texttt{coordinates}` is a list of longitude-latitude pairs, `\texttt{timestamps}` is optional and provides time information for each point (in Unix time), and `\texttt{radiuses}` specifies the positional uncertainty in meters.

`\subsection{Response Structure}`

The response contains a series of `\texttt{matchings}`, each of which includes a snapped geometry and additional metadata. Key fields include:

```
\begin{itemize}
  \item \texttt{matchings[*].geometry}: A GeoJSON LineString of the matched route.
  \item \texttt{tracepoints}: An array of points indicating how each GPS observation was
    matched.
  \item \texttt{matchings[*].legs[*].annotation.nodes}: A list of OSM node IDs representing
    the matched path.
\end{itemize}
```

Note that OSRM does not return OpenStreetMap `\texttt{way\_id}`s directly. However, with additional preprocessing or Overpass API queries, these node sequences can be translated back into way identifiers if needed.

`\subsection{Python Integration}`

The following Python function sends a GPS trace to the OSRM `\texttt{/match}` service:

```
\begin{lstlisting}[language=Python]
import requests

def osrm_match(points, timestamps=None, host="http://localhost:5000", profile="bicycle"):
    coords = [[lon, lat] for lat, lon in points]
    url = f"{host}/match/v1/{profile}"
    body = {
        "coordinates": coords,
        "geometries": "geojson",
        "overview": "full",
        "steps": False
    }
    if timestamps:
        body["timestamps"] = timestamps
    response = requests.post(url, json=body)
    return response.json()
\end{lstlisting}
```

This pipeline enables full reconstruction of a matched route from raw GPS input, suitable for further analysis such as segment extraction, elevation profiling, or training suitability scoring.

`\subsection{Segmentation}`

`\subsection{Categorization}`

`\subsection{Segment Analysis}`

`\subsubsection{Wavelet Transform}`

Sharifzadeh et al. (2005) propose the use of the Wavelet Transform Function, and more specifically Wavelet Footprints,

to address the problem of change detection in time series data.

Wavelet Decomposition of Trajectory: Given a signal $x[n]$ of length N , it can be expressed as a wavelet series:

$$x[n] = \sum_{j=0}^{J-1} \sum_k d_{j,k} \psi_{j,k}[n] + \sum_k c_{J,k} \phi_{J,k}[n]$$

where:

$\psi_{j,k}[n]$ are the wavelet basis functions at scale j and position k .

$\phi_{J,k}[n]$ are the scaling functions at the coarsest level J .

$d_{j,k}$ are detail coefficients at scale j and position k .

$c_{J,k}$ are approximation coefficients.

Footprint Definition:

The **wavelet footprint** of x is defined as the set of significant coefficients:

$$F(x) = \{ (j,k) \mid |d_{j,k}| > T_j \}$$

where T_j is a scale-dependent threshold, which can be tuned to correspond to physiological thresholds of training intensity zones. This is useful since at different scales, the detail coefficients are not comparable, so the threshold must be defined explicitly.

Footprint Calculation:

To calculate the Wavelet Footprint, first the detail coefficient, $d_{j,k}$ is extracted. Since not all detail coefficients are meaningful, a scale-dependent threshold T_j is defined to filter out noise. This is calculated with the standard deviation of the specific scale:

$$T_j = \gamma_j \sigma_j$$

Where:

σ_j is the standard deviation of $d_{j,k}$ at scale j .

γ_j is the tuning parameter.

This value can then be used to define the Footprint Definition described earlier.

Compact Representation:

To reduce dimensionality, only the k largest coefficients can be retained:

$$F_k(x) = \text{top-}k \text{ elements of } F(x)$$

```

168         This is useful to optimize data set size, while not reducing the accuracy of the
169         data significantly.
170         \item \textbf{Matching and Distance}\\
171         The overall difference in two signals  $x$  and  $y$  can be approximated by taking the
172         difference of their footprints:
173         \[
174             D(x,y) \approx |F(x) - F(y)|
175         \]
176     \end{itemize}
177     \subsubsection{Normalized Power}
178     Applying Normalized Power is a useful metric for determining variability in power data. The longer
179     the signal, the more useful NP would be, so
180     we can apply a weighting inversely proportional to the average duration of the segment.
181     However, as the time gets shorter, we can mitigate the accuracy loss by tapering off the 30 second
182     rolling average towards the end of the segment.
183     \\
184     Instead of a 30 second rolling average, we will take the lesser of either 30 seconds, or the
185     remaining duration of the segment, so we don't end up
186     with a lot of segments with skewed NP due to the rolling window running off the edge of the data
187     set.\\
188     We will define a variable  $k$  that represents this:
189     \[
190         k[t] = \begin{cases}
191             (T-t), & \& (T-t) < 30 \\
192             30, & \& (T-t) \geq 30
193         \end{cases}
194     \]
195     Now we can define the rolling average,  $\overline{P}_r$  at time  $t$  to be:
196     \[
197         \overline{P}_r[t] = \frac{1}{k[t]} \sum_{j=0}^{k[t]-1} P[j]
198     \]
199     The final definition of NP is:
200     \[
201         \mathrm{NP} = \frac{1}{M} \sum_{i=1}^M \big(\overline{P}_r[i]\big)^4 \bigg)^{1/4}
202     \]
203     \subsubsection{Training Suitability Score Evaluation}
204     \end{document}

```